

BÁO CÁO DỰ ÁN

Hệ thống SecureChat

NGUYỄN VŨ THÀNH

Thanh14704@gmail.com

Lĩnh Vực: Kỹ Sư Phần Mềm: Lập Trình Hệ Thống

Mentor hướng dẫn: Lê Thành Thân

Đơn vị: Viettel Cypher Security

Chương trình: Viettel Digital Talent

HÀ NỘI, 6/2026

TÓM TẮT NỘI DUNG DỰ ÁN

Dự án trình bày việc thiết kế và phát triển hệ thống SecureChat, một giải pháp truyền thông thời gian thực (Real-time Communication) dựa trên mô hình Client-Server. Hệ thống được xây dựng bằng ngôn ngữ C++ nhằm đáp ứng hai yêu cầu cốt lõi: khả năng xử lý đồng thời (High Concurrency) và bảo mật thông tin đa lớp.

Ở phía Máy chủ (Server), kiến trúc được thiết kế theo mô hình điều khiển sự kiện bất đồng bộ (Event-Driven) thông qua cơ chế *epoll* kết hợp với Thread Pool. Cấu trúc này hỗ trợ giải quyết bài toán C10K và tối ưu hóa thông lượng tại các luồng I/O mạng. Cơ sở dữ liệu SQLite được triển khai cùng cơ chế Write-Ahead Logging (WAL) nhằm duy trì tính nhất quán và tốc độ truy xuất trong môi trường đa luồng.

Về giao thức và bảo mật, hệ thống ứng dụng giao thức truyền tải nhị phân tùy chỉnh (Custom Binary Protocol) có tích hợp mã kiểm lỗi CRC32 tại Header. Cơ chế mã hóa lai (Hybrid Crypto) được áp dụng, sử dụng RSA-2048 để thiết lập khóa phiên an toàn (Handshake) và AES-256-GCM để mã hóa, đồng thời xác thực tính nguyên vẹn của toàn bộ tải trọng thông điệp (Payload).

Bên cạnh đó, dự án tích hợp các phân hệ an ninh mạng chủ động, bao gồm Hệ thống phát hiện xâm nhập (Intrusion Detector - IDS) để phân tích hành vi bất thường, Bộ giới hạn tốc độ (Rate Limiter) dựa trên thuật toán Token-Bucket nhằm phòng chống tấn công DoS/Spam, và hệ thống Nhật ký kiểm toán (Audit Logger) để lưu vết các sự kiện bảo mật. Phía Máy khách (Client) triển khai giao diện dòng lệnh (CLI), hỗ trợ đa dạng nghiệp vụ từ nhắn tin cá nhân, phòng trò chuyện nhóm, trung chuyển tệp tin ẩn danh, đến các luồng thực thi lệnh quản trị (Admin Commands) dựa trên cơ sở phân quyền (RBAC).

Kết quả nghiên cứu cung cấp một nền tảng giao tiếp hoạt động ổn định, tuân thủ các tiêu chuẩn bảo mật hiện hành và đảm bảo khả năng mở rộng kiến trúc hệ thống trong tương lai.

MỤC LỤC

Danh sách hình vẽ	iii
Danh sách bảng	iv
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Khảo sát hiện trạng	1
1.1.1 Đặt vấn đề	1
1.1.2 Mục tiêu và phạm vi của đề tài	1
1.2 Phân tích yêu cầu hệ thống	2
1.2.1 Yêu cầu chức năng	2
1.2.2 Yêu cầu phi chức năng	3
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	5
2.1 Kiến trúc tổng thể và Phân tích yêu cầu	5
2.1.1 Phân tích yêu cầu hệ thống	5
2.1.2 Kiến trúc tổng thể	6
2.2 Giao thức Mạng và Mô hình I/O	8
2.2.1 Mô hình Client-Server và I/O Multiplexing	8
2.2.2 Thiết kế Định dạng Gói tin	9
2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực	11
2.3.1 Cấu trúc Lưu trữ Dữ liệu	11
2.3.2 Luồng Xác thực và Quản lý Phiên	12
2.4 Giao thức Mật mã và Tính bảo mật	14
2.4.1 Luồng Trao đổi khóa	14
2.5 Hệ thống Chức năng Cốt lõi	15
2.5.1 Quản lý Phòng và Định tuyến Thông điệp	15
2.5.2 Trung chuyển Tập tin Ẩn danh	17
2.6 Hệ thống Quản trị và Chống lạm dụng	19
2.6.1 Kiểm soát Rủi ro Tấn công Từ chối Dịch vụ (DoS/DDoS)	19
2.6.2 Kiểm soát Truy cập và Quản trị (RBAC)	21
CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	24

3.1	Môi trường và Công nghệ Triển khai	24
3.1.1	Cấu hình Môi trường Hệ thống	24
3.1.2	Kịch bản Biên dịch và Vận hành	25
3.2	Kịch bản Kiểm thử và Đánh giá Hiệu năng	28
3.2.1	Tổng quan Hệ thống Kiểm thử Tự động	28
3.2.2	Đánh giá Giới hạn Chịu tải (Load Benchmark)	30
3.3	Đánh giá Khả năng Phòng thủ Hệ thống	31
3.3.1	Phòng thủ trước tấn công Từ chối dịch vụ	31
3.3.2	Phòng thủ trước tấn công Tiêm nhiễm và Vượt quyền	33
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		35
4.1	Kết quả đạt được	35
4.2	Hướng phát triển tương lai	36

Danh sách hình vẽ

Hình 2.1:	Sơ đồ Component Kiến trúc Tổng thể Hệ thống	8
Hình 2.2:	Sơ đồ Thực thể Liên kết của hệ thống CSDL	12
Hình 2.3:	Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên	13
Hình 2.4:	Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES	15
Hình 2.5:	Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng	18
Hình 2.6:	Máy trạng thái hữu hạn luồng Truyền tệp ẩn danh	20
Hình 2.7:	Lược đồ Khối Hệ thống Tường lửa mềm	22

Danh sách bảng

Bảng 2.1:	Đặc tả yêu cầu chức năng hệ thống	5
Bảng 2.2:	Đặc tả yêu cầu phi chức năng hệ thống	6
Bảng 2.3:	Phân rã các thành phần kiến trúc hệ thống	6
Bảng 2.4:	Cấu trúc Gói tin Nhị phân	10
Bảng 2.5:	Đặc tả các Bảng thực thể trong Cơ sở dữ liệu	11
Bảng 2.6:	Chiến lược cấp phát khóa trong RoomManager	17
Bảng 2.7:	Các trạng thái của Phiên truyền tệp	19
Bảng 2.8:	Hệ thống Điểm phạt Hành vi	21
Bảng 2.9:	Ma trận Phân quyền của Hệ thống	23
Bảng 3.1:	Bảng Ma trận Phụ thuộc Thư viện	25
Bảng 3.2:	Các tập lệnh và tham số cấu hình trong quy trình biên dịch	26
Bảng 3.3:	Các tham số dòng lệnh vận hành Máy chủ	27
Bảng 3.4:	Các tham số dòng lệnh Máy khách	28
Bảng 3.5:	Ma trận phân bổ các module kiểm thử tự động	29
Bảng 3.6:	Đo lường Thông lượng và Độ trễ của Server	31
Bảng 3.7:	Cấu hình giới hạn tốc độ của hệ thống	33
Bảng 3.8:	Ma trận điểm số rủi ro của hệ thống phòng thủ	34

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
ACK	Acknowledgment
AD	Authenticated Data
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
CLI	Command-Line Interface
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DoS	Denial-of-Service
DDoS	Distributed Denial-of-Service
ERD	Entity-Relationship Diagram
ET	Edge-Triggered
FD	File Descriptor
GCM	Galois/Counter Mode
GUI	Graphical User Interface
HIDS	Host-based Intrusion Detection System
HMAC	Hash-based Message Authentication Code
I/O	Input/Output
IDS	Intrusion Detection System
IP	Internet Protocol
LT	Level-Triggered
MTU	Maximum Transmission Unit
OOM	Out of Memory
RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAL	Write-Ahead Logging

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Khảo sát hiện trạng

1.1.1 Đặt vấn đề

Máy chủ trò chuyện (Chat Server) đóng vai trò trung tâm trong hệ thống truyền thông thời gian thực (Real-time Communication), đảm nhiệm chức năng định tuyến thông điệp song công (Full-duplex) giữa các máy khách (Client).

Trong điều kiện xử lý tải cao, mô hình cấp phát luồng theo kết nối (Thread-per-client) không đáp ứng hiệu quả bài toán đa kết nối đồng thời (C10K), dẫn đến thắt cổ chai I/O (I/O Bottleneck) và suy giảm hiệu suất sử dụng tài nguyên. Bên cạnh đó, việc sử dụng giao thức văn bản (Text-based Protocol) phát sinh chi phí phân tích cú pháp (Parsing overhead) và gia tăng dung lượng truyền tải. Để khắc phục các vấn đề này, giải pháp kết hợp mô hình I/O đa kênh (I/O Multiplexing) và giao thức nhị phân (Binary Protocol) được áp dụng nhằm tối ưu hóa băng thông và giảm thiểu độ trễ.

Bảo vệ an toàn dữ liệu truyền tải là thành phần thiết yếu của hệ thống. Sự thiếu hụt cơ chế kiểm soát truy cập và bảo vệ kênh truyền làm tăng nguy cơ từ các hình thức tấn công nghe lén (Eavesdropping), tấn công xen giữa (MitM) và từ chối dịch vụ (DoS/Spam). Do đó, hệ thống yêu cầu tích hợp các thành phần bảo mật bao gồm cơ chế mã hóa dữ liệu, hệ thống phát hiện xâm nhập (IDS) và giới hạn lưu lượng (Rate Limiter).

Dựa trên cơ sở đó, nghiên cứu này tập trung phát triển hệ thống máy chủ trò chuyện an toàn (Secure Chat Server) hiệu năng cao, hướng tới tối ưu hóa I/O mạng, quá trình định tuyến thông điệp và triển khai cơ chế bảo mật toàn diện.

1.1.2 Mục tiêu và phạm vi của đề tài

1. Mục tiêu của đề tài

Đề tài nghiên cứu và phát triển hệ thống máy chủ truyền thông (Secure Chat Server) theo mô hình Client-Server với các mục tiêu cụ thể:

- Tối ưu hóa hiệu năng: Triển khai cơ chế I/O Multiplexing (epoll) kết hợp Thread Pool để giải quyết bài toán C10K, cải thiện thông lượng xử lý I/O và giảm độ trễ hệ thống.
- Tối ưu hóa giao thức: Áp dụng giao thức nhị phân tùy chỉnh (Custom Binary Protocol) nhằm loại bỏ chi phí phân tích cú pháp và tối thiểu hóa băng thông mạng.
- Đảm bảo an toàn thông tin: Tích hợp cơ chế mã hóa lai (RSA-2048, AES-256-GCM), hệ thống phát hiện xâm nhập (IDS), thuật toán giới hạn lưu lượng

(Token-Bucket Rate Limiter) và ghi log kiểm toán (Audit Logger).

- Đảm bảo tính nhất quán dữ liệu: Tích hợp cơ sở dữ liệu SQLite cùng cơ chế Write-Ahead Logging (WAL) để tối ưu hóa hiệu suất truy vấn trong môi trường đa luồng.

2. Phạm vi nghiên cứu và triển khai

Phạm vi nghiên cứu và giới hạn của đề tài bao gồm:

- Kiến trúc hệ thống: Lõi máy chủ (Server-side) được lập trình bằng ngôn ngữ C++. Ứng dụng máy khách (Client-side) được triển khai dưới dạng giao diện dòng lệnh (CLI) phục vụ mục đích kiểm thử (giới hạn không bao gồm giao diện người dùng đồ họa hoặc ứng dụng di động).
- Chức năng nghiệp vụ: Hỗ trợ tính năng nhắn tin cá nhân (Direct Message), trò chuyện nhóm (Rooms), truyền tải tệp tin (File Transfer) và tích hợp các tập lệnh quản trị (Admin Commands).
- Đánh giá hệ thống: Thực hiện đo lường hiệu năng (Benchmark) dưới tải cao và mô phỏng các kịch bản tấn công (DoS, quét cổng, giả mạo gói tin) nhằm đánh giá năng lực phòng thủ.

1.2 Phân tích yêu cầu hệ thống

1.2.1 Yêu cầu chức năng

Hệ thống yêu cầu đáp ứng các nhóm chức năng cốt lõi bao gồm:

1. Quản lý tài khoản và xác thực (Authentication & Authorization):

- Xử lý đăng ký (Register) và đăng nhập (Login). Mật khẩu được băm (Hash) bằng thuật toán chuẩn (SHA256/bcrypt) trước khi lưu trữ vào cơ sở dữ liệu.
- Cấp phát thẻ định danh phiên (Session Token) thông qua mã xác thực HMAC, cung cấp khả năng tự động phục hồi kết nối (Session Resumption).

2. Quản lý phiên kết nối và trạng thái (Session Management):

- Quản lý vòng đời kết nối mạng (Connect, Disconnect) và duy trì tín hiệu kiểm tra trạng thái hoạt động (Ping/Pong).
- Cung cấp chức năng tra cứu danh sách người dùng trực tuyến (List Users) và thông tin hồ sơ (Whois).

3. Giao tiếp và nhắn tin thời gian thực (Real-time Chatting):

- Nhắn tin cá nhân (Direct Message): Trao đổi thông điệp trực tiếp giữa hai máy khách (1-1).
- Nhắn tin nhóm (Room Chat): Cung cấp tính năng liệt kê phòng (List Rooms), tạo phòng (Create Room), tham gia (Join), rời phòng (Leave) và phát đa hướng thông điệp (Broadcast).
- Lưu trữ và truy vấn lịch sử trò chuyện thông qua hệ quản trị cơ sở dữ liệu.

4. Truyền tải tệp tin (File Transfer):

- Quản lý luồng điều khiển truyền tệp hai chiều thông qua các thông điệp: gửi yêu cầu (File Request) và phản hồi (Accept/Reject).
- Truyền tải dữ liệu dưới dạng phân mảnh (Chunking) nhằm duy trì thông lượng cho luồng thông điệp chính.

5. Quản trị hệ thống (Admin Commands):

- Tích hợp cơ chế phân quyền quản trị viên (Admin Privilege).
- Thực thi các lệnh điều khiển và xử lý vi phạm: kích xuất (Kick), cấm gửi tin (Mute), cấm truy cập (Ban IP/Username), thăng cấp (Promote) và phát thông báo hệ thống (System Broadcast).

6. Bảo mật hệ thống (Security):

- Bắt tay mật mã (Crypto Handshake): Thực hiện trao đổi khóa (Key Exchange) trong quá trình khởi tạo kết nối nhằm thiết lập kênh truyền mã hóa.
- Giới hạn lưu lượng (Rate Limiting): Hạn chế tần suất gửi thông điệp để giảm thiểu nguy cơ tấn công từ chối dịch vụ (DoS) và thư rác (Spam).
- Hệ thống phát hiện xâm nhập (IDS): Nhận diện và ghi log các vi phạm (đăng nhập sai vượt giới hạn, quét cổng, gói tin dị dạng) làm cơ sở cho biện pháp khóa IP (Ban).

1.2.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng tập trung vào cấu trúc hệ thống, hiệu suất xử lý và cơ chế bảo mật:

1. Hiệu năng và khả năng mở rộng (Performance & Scalability):

- Xử lý đồng thời: Áp dụng kiến trúc I/O đa kênh (I/O Multiplexing) với cơ chế epoll kết hợp Thread Pool nhằm vận hành luồng điều phối phi đồng bộ (Non-blocking). Hệ thống yêu cầu đáp ứng ngưỡng kết nối C10K trong điều kiện độ trễ nằm trong khoảng cho phép.
- Tối ưu hóa băng thông: Triển khai giao thức nhị phân tùy chỉnh (Custom Binary Protocol) nhằm tối giản kích thước gói tin và giảm chi phí phân tích cú pháp (Parsing).

2. Bảo mật và toàn vẹn dữ liệu (Security & Integrity):

- Mã hóa kênh truyền: Thực thi mã hóa tải trọng dữ liệu bằng thuật toán AES-256-GCM và trao đổi khóa phiên (Session Key) thông qua thuật toán RSA-2048.
- Toàn vẹn thông điệp: Kiểm soát tính toàn vẹn thông qua hàm kiểm lỗi CRC32 và mã xác thực HMAC được đính kèm trong mỗi gói tin.
- Lưu trữ dữ liệu: Sử dụng hàm băm một chiều cho dữ liệu định danh. Triển khai cơ chế Write-Ahead Logging (WAL) của cơ sở dữ liệu SQLite nhằm bảo đảm

tính nhất quán của dữ liệu lưu trữ.

3. Tính tin cậy và khả năng chịu lỗi (Reliability & Fault Tolerance):

- Quản lý tài nguyên: Hệ thống áp dụng cơ chế tự động thu hồi tài nguyên (bao gồm socket, phiên đăng nhập, và vùng nhớ được cấp phát) đối với các kết nối vượt quá thời gian chờ (Timeout) hoặc bị ngắt kết nối đột ngột. Quá trình này nhằm tránh tình trạng cạn kiệt bộ mô tả tệp (File Descriptor Exhaustion) và rò rỉ bộ nhớ (Memory Leak).
- Xử lý luồng mạng: Phân mảnh và tái tổ hợp gói tin mạng thông qua việc cấp phát bộ đệm (Network Buffer) độc lập trên từng phiên kết nối, nhằm giải quyết đặc tính phân mảnh (TCP Fragmentation) và gộp gói (TCP Sticky Packets) của giao thức TCP.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày chi tiết kiến trúc tổng thể và các quyết định thiết kế cốt lõi của hệ thống. Quá trình phân tích và thiết kế được gắn kết chặt chẽ với các nền tảng lý thuyết về mạng máy tính, mô hình vào/ra (I/O) và mật mã học, nhằm làm rõ cơ chế vận hành của từng phân hệ (module) trong kiến trúc thực tế.

2.1 Kiến trúc tổng thể và Phân tích yêu cầu

2.1.1 Phân tích yêu cầu hệ thống

Mục này xác định các tiêu chuẩn thiết kế cho hệ thống máy chủ truyền thông. Yêu cầu hệ thống bao gồm hai nhóm chính: chức năng và phi chức năng.

1. Yêu cầu chức năng

Bảng 2.1 đặc tả các chức năng đối với Người dùng (User) và Quản trị viên (Admin).

Bảng 2.1: Đặc tả yêu cầu chức năng hệ thống

Nhóm chức năng	Tác nhân	Mô tả chi tiết
Xác thực định danh	Người dùng	Đăng ký, đăng nhập và quản lý vòng đời phiên làm việc.
Nhắn tin trực tiếp	Người dùng	Truyền thông điệp song công (Full-duplex) 1-1.
Quản lý phòng trò chuyện	Người dùng	Khởi tạo, tham gia và quản lý trạng thái phòng trò chuyện.
Truyền tải tệp tin	Người dùng	Truyền nhận tệp đa phương tiện thông qua cơ chế phân mảnh (Data Chunking).
Quản trị hệ thống	Quản trị viên	Kiểm soát phân quyền (RBAC), quản lý tài khoản và giám sát hệ thống.
Xử lý vi phạm	Quản trị viên	Thực thi các thao tác: cấm gửi tin, ngắt kết nối và giới hạn tài khoản.

2. Yêu cầu phi chức năng

Bảng 2.2 đặc tả các ràng buộc kỹ thuật về hiệu suất, toàn vẹn dữ liệu và an toàn thông tin.

Bảng 2.2: Đặc tả yêu cầu phi chức năng hệ thống

Tiêu chí	Mô tả chi tiết
Hiệu năng và Xử lý đồng thời	Duy trì tính khả dụng và kiểm soát độ trễ. Quản lý kết nối quy mô lớn (C10K) thông qua cơ chế I/O phi đồng bộ kết hợp Thread Pool và Event Loop.
Bảo mật và Mã hóa	Bảo mật luồng dữ liệu truyền tải thông qua cơ chế mã hóa đối xứng AES kết hợp trao đổi khóa RSA.
An toàn mạng	Kiểm soát giới hạn lưu lượng nhằm giảm thiểu rủi ro từ các hình thức tấn công DoS/DDoS, Brute-force và Spam.
Nhật ký và Kiểm toán	Lưu trữ nhật ký hệ thống nhằm hỗ trợ quá trình kiểm toán và truy vết.

2.1.2 Kiến trúc tổng thể (System Architecture)

Kiến trúc máy chủ ứng dụng mô hình phân tầng (Layered Architecture) và nguyên lý tách biệt trách nhiệm (Separation of Concerns). Hệ thống được cấu trúc thành 3 phân tầng độc lập, phụ trách xử lý từ cấp độ mạng thô đến logic nghiệp vụ.

Bảng 2.3 mô tả chi tiết chức năng của các thành phần trong từng phân tầng.

Bảng 2.3: Phân rã các thành phần kiến trúc hệ thống

Phân tầng	Thành phần	Chức năng cốt lõi
Tầng Mạng và Cốt lõi	TcpServer	Quản lý kết nối TCP.
	EventLoop	Điều phối sự kiện mạng bất đồng bộ.
	ThreadPool	Xử lý khối lượng tính toán (CPU-bound).
	ClientSession	Duy trì trạng thái phiên kết nối.
Tầng Bảo mật và Lọc	KeyExchange	Thực thi trao đổi khóa phiên.
	CryptoEngine	Mã hóa và giải mã luồng dữ liệu.
	RateLimiter	Kiểm soát giới hạn tần suất truy cập.

Phân tầng	Thành phần	Chức năng cốt lõi
Tầng Nghiệp vụ	IntrusionDetector	Phát hiện dấu hiệu tấn công.
	MessageFilter	Kiểm duyệt tính hợp lệ của dữ liệu đầu vào.
	AuditLogger	Ghi nhận sự kiện bảo mật.
	AuthManager	Xác thực và quản lý định danh người dùng.
	RoomManager	Quản lý vòng đời phòng trò chuyện.
	Room	Quản lý trạng thái thành viên và đa hướng thông điệp.
	FileTransfer	Điều phối truyền tải và phân mảnh dữ liệu.
	AdminCommands	Thực thi tập lệnh điều khiển của quản trị viên.

Cơ chế tương tác giữa các phân tầng được định nghĩa như sau:

1. Tầng Mạng và Cốt lõi

Tương tác với Socket API theo mô hình phi đồng bộ nhằm cung cấp phương thức xử lý I/O mạng.

- TcpServer: Lắng nghe kết nối TCP và cấp phát tài nguyên hệ thống.
- EventLoop: Điều phối sự kiện thông qua cơ chế I/O đa kênh.
- ThreadPool: Tiếp nhận và thực thi các tác vụ tính toán.
- ClientSession: Quản lý dữ liệu trạng thái, cấu trúc bộ đệm và tham chiếu khóa mật mã của phiên kết nối.

2. Tầng Bảo mật và Lọc

Kiểm soát và tiền xử lý dữ liệu trước khi chuyển giao quyền kiểm soát cho tầng nghiệp vụ.

- KeyExchange & CryptoEngine: Quản lý giao thức trao đổi khóa và thuật toán mã hóa/giải mã.
- RateLimiter: Giám sát và giới hạn tần suất yêu cầu mạng.
- IntrusionDetector & MessageFilter: Nhận diện dấu hiệu bất thường và loại bỏ luồng dữ liệu không hợp lệ.

- AuditLogger: Thu thập và lưu trữ thông tin sự kiện an ninh.

3. Tầng Nghiệp vụ

Xử lý và thực thi logic ứng dụng.

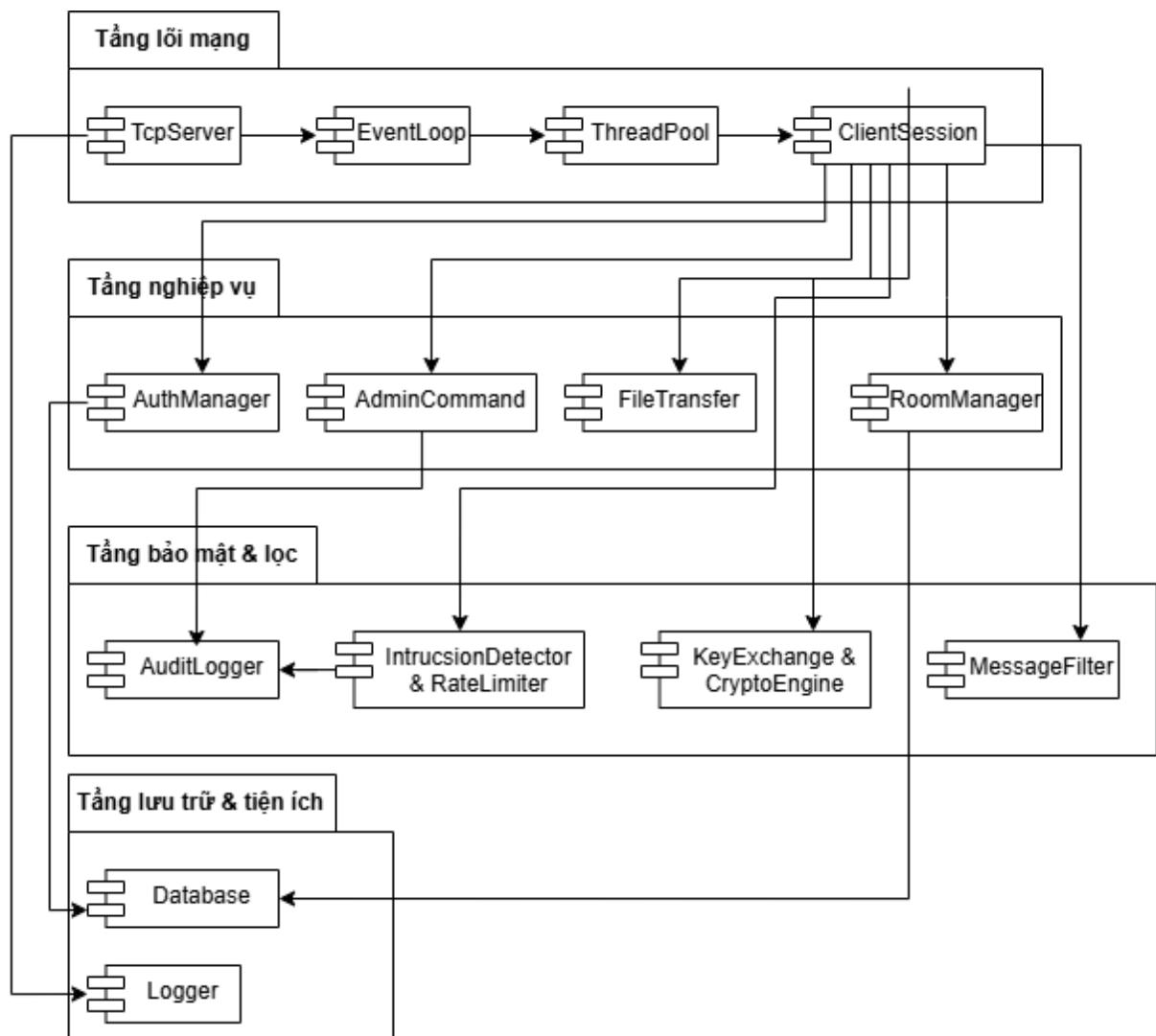
- AuthManager: Kiểm chứng thông tin định danh và vòng đời phiên làm việc.
- RoomManager & Room: Kiểm soát cấu trúc phòng và định tuyến thông điệp giữa các thành viên.
- FileTransfer: Tổ chức truyền tải, phân mảnh và tái tổ hợp dữ liệu nhị phân.
- AdminCommands: Cung cấp giao diện thực thi lệnh quản trị.

2.2 Giao thức Mạng và Mô hình I/O

2.2.1 Mô hình Client-Server và I/O Multiplexing (epoll)

1. Cơ sở lý thuyết

Mô hình cấp phát luồng trên mỗi kết nối (Thread-per-connection) đòi hỏi chi phí



Hình 2.1: Sơ đồ Component Kiến trúc Tổng thể Hệ thống

chuyển đổi ngữ cảnh (Context Switching) cao đối với bài toán C10K. Do đó, kỹ thuật I/O Multiplexing với API `epoll` (đặc tả trên hệ điều hành Linux) được triển khai nhằm hỗ trợ một luồng đơn giám sát đồng thời nhiều bộ mô tả tệp (File Descriptor - FD). Thông qua cấu trúc cây đỏ đen (Red-Black Tree) và danh sách liên kết (Linked List), `epoll` tối ưu hóa quá trình thu thập sự kiện với độ phức tạp $\mathcal{O}(1)$.

2. Triển khai hệ thống

Thành phần `TcpServer` ứng dụng I/O phi đồng bộ (Non-blocking I/O) và `epoll` nhằm điều phối luồng kết nối TCP theo quy trình sau:

- Khởi tạo Socket lắng nghe: Chuyển đổi trạng thái Socket sang chế độ phi đồng bộ thông qua hàm `fcntl`.
- Khởi tạo I/O Multiplexer: Tạo đối tượng `epoll` (thông qua `epoll_create1`) và đăng ký Socket lắng nghe vào hàng đợi sự kiện với cờ `EPOLLIN` (thông qua `epoll_ctl`).
- Vòng lặp điều phối sự kiện: Gọi hàm `epoll_wait` để thu thập các sự kiện I/O khả dụng.
 - Xử lý kết nối mới: Thực thi `accept`, chuyển đổi `clientFd` sang trạng thái phi đồng bộ, đăng ký bộ mô tả tệp vào `epoll` (với cờ `EPOLLIN | EPOLLRDHUP`) và cấp phát đối tượng `ClientSession`.
 - Xử lý luồng dữ liệu: `TcpServer` tiếp nhận chuỗi byte thô (thông qua hàm `recv`) và giao phó cho `ClientSession` để thực thi phân giải.

2.2.2 Thiết kế Định dạng Gói tin (Packet Format)

1. Cơ sở lý thuyết

TCP là giao thức hướng luồng (Stream-oriented), đặc tính này thường phát sinh hiện tượng dính gói (Sticky Packets) và phân mảnh (TCP Fragmentation). Tầng ứng dụng áp dụng giao thức nhị phân (Binary Protocol) nhằm xác định chính xác ranh giới gói tin, tối ưu hóa băng thông mạng và giảm thiểu chi phí phân tích cú pháp (Parsing Overhead).

2. Cấu trúc gói tin

Hệ thống sử dụng định dạng gói tin nhị phân bao gồm Header (17 Bytes) và Payload, cấu trúc chi tiết được trình bày tại Bảng 2.4.

Bảng 2.4: Cấu trúc Gói tin Nhị phân

Phân đoạn	Trường dữ liệu	Kích thước	Mô tả chi tiết
Header	magic (Nhận diện)	2 Bytes	Chuỗi byte xác thực định dạng gói tin.
	version (Phiên bản)	1 Byte	Định danh phiên bản giao thức hệ thống.
	msg_type (Loại tin)	1 Byte	Phân loại thông điệp nghiệp vụ.
	flags (Cờ điều khiển)	1 Byte	Cờ trạng thái gói tin (mã hóa, nén).
	sequence_num (Thứ tự)	4 Bytes	Đánh dấu thứ tự nhằm hỗ trợ quá trình tái tổ hợp.
	payload_length (Kích thước)	4 Bytes	Xác định kích thước nhằm phân tách ranh giới gói tin.
	checksum (Mã kiểm tra)	4 Bytes	Kiểm định tính toàn vẹn dữ liệu (Integrity).
Payload	Dữ liệu JSON	Tùy biến	Chứa cấu trúc dữ liệu nghiệp vụ (Mã hóa AES-256).

Quá trình đóng gói và phân giải thông điệp (Packet Parsing) bao gồm các công đoạn:

- Đóng gói dữ liệu: Thực hiện tuần tự hóa cấu trúc JSON, mã hóa tải trọng và đính kèm Header (17 Bytes) vào đầu gói tin.
- Phân tách Header: Đối tượng `ClientSession` đọc 17 bytes đầu tiên nhằm trích xuất các tham số điều khiển (`msg_type`, `payload_length`, `magic`, `checksum`).
- Đọc tải trọng (Payload): Cấp phát vùng nhớ tương ứng với `payload_length` nhằm tiếp nhận dữ liệu và xử lý hiện tượng phân mảnh luồng TCP.
- Giải mã và định tuyến: `CryptoEngine` thực thi giải mã dữ liệu, tiếp đó hệ thống dựa trên tham số `msg_type` để định tuyến thông điệp đến module nghiệp vụ tương ứng.

2.3 Thiết kế Cơ sở dữ liệu và Cơ chế Xác thực

2.3.1 Cấu trúc Lưu trữ Dữ liệu

1. Cơ sở lý thuyết

Hệ quản trị CSDL quan hệ (RDBMS) duy trì tính toàn vẹn thông qua các thuộc tính ACID. Lược đồ dữ liệu (Database Schema) tuân thủ dạng chuẩn 3 (3NF) nhằm ngăn ngừa dị thường cập nhật (Update Anomalies). Cơ chế Write-Ahead Logging (WAL) được áp dụng để hỗ trợ xử lý truy vấn đọc/ghi đồng thời (Concurrency) theo dạng phi chặn (Non-blocking).

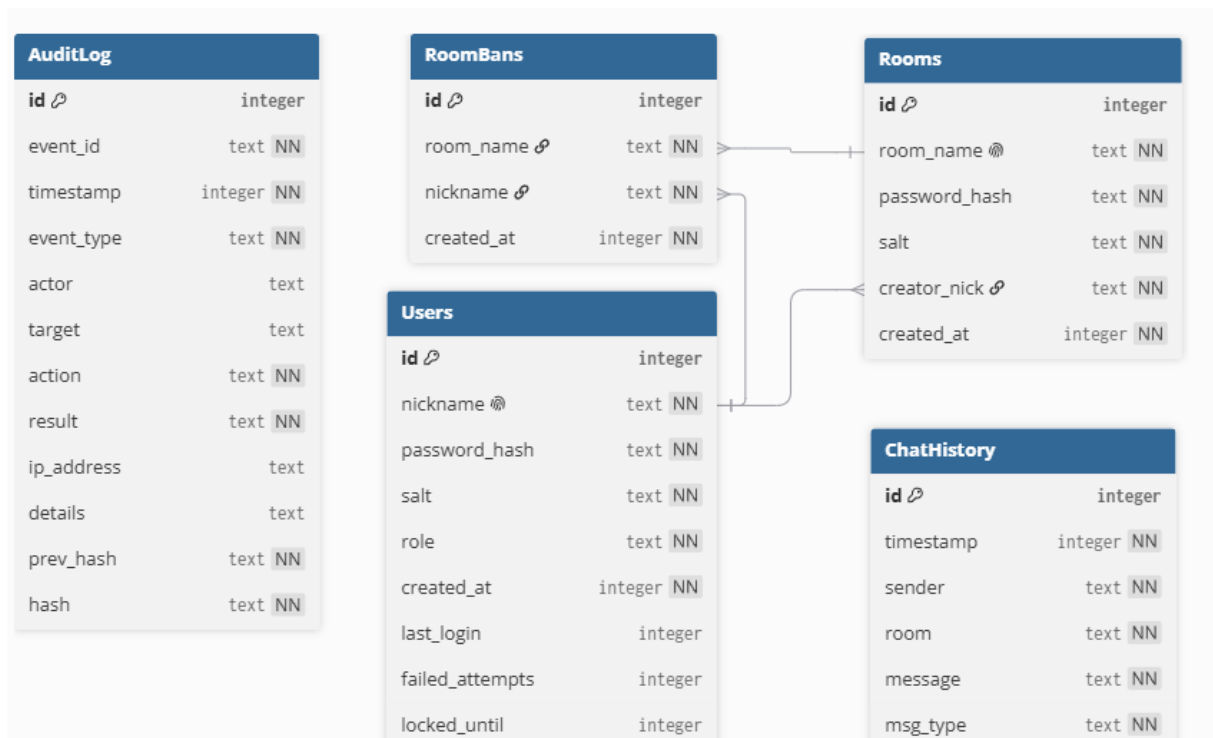
2. Triển khai hệ thống

Hệ thống sử dụng SQLite3 (RDBMS nhúng) nhằm giảm thiểu độ trễ giao tiếp liên tiến trình (IPC Overhead). Các thiết lập `PRAGMA journal_mode=WAL` và `PRAGMA foreign_keys=ON` lần lượt hỗ trợ truy xuất đồng thời và thực thi tính toàn vẹn tham chiếu (Referential Integrity). Bảng 2.5 trình bày chi tiết lược đồ dữ liệu với 5 bảng thực thể.

Bảng 2.5: Đặc tả các Bảng thực thể trong Cơ sở dữ liệu

Tên Bảng	Khóa & Chỉ mục (Index)	Mô tả chi tiết và Vai trò cốt lõi
Users	PRIMARY KEY (id) UNIQUE (nickname)	Quản lý thông tin định danh và tham số băm mật khẩu (password_hash, salt). Hỗ trợ kiểm soát tấn công Brute-force thông qua trường failed_attempts và locked_until.
Rooms	PRIMARY KEY (id) UNIQUE (room_name)	Lưu trữ siêu dữ liệu phòng, bao gồm thông tin xác thực (mật khẩu băm). Cấu trúc khóa ngoại creator_nick tham chiếu thực thể Users kết hợp ràng buộc ON DELETE CASCADE.
RoomBans	PRIMARY KEY (id) idx_roombans	Quản lý danh sách cấm (Blacklist) của phòng. Ràng buộc UNIQUE(room_name, nickname) đảm bảo tính duy nhất của dữ liệu.

Tên Bảng	Khóa & Chỉ mục (Index)	Mô tả chi tiết và Vai trò cốt lõi
ChatHistory	PRIMARY KEY (id) idx_chat_room_ts	Lưu trữ thông điệp mạng. Triển khai chỉ mục liên hợp (Composite Index) trên (room, timestamp) nhằm tối ưu hóa tốc độ truy xuất.
AuditLog	PRIMARY KEY (id) idx_audit_ts	Quản lý nhật ký kiểm toán. Áp dụng chuỗi băm (Hash Chain) qua trường prev_hash để đảm bảo tính toàn vẹn dữ liệu (Tamper-evident).



Hình 2.2: Sơ đồ Thực thể Liên kết (ERD) của hệ thống CSDL

2.3.2 Luồng Xác thực và Quản lý Phiên (AuthManager)

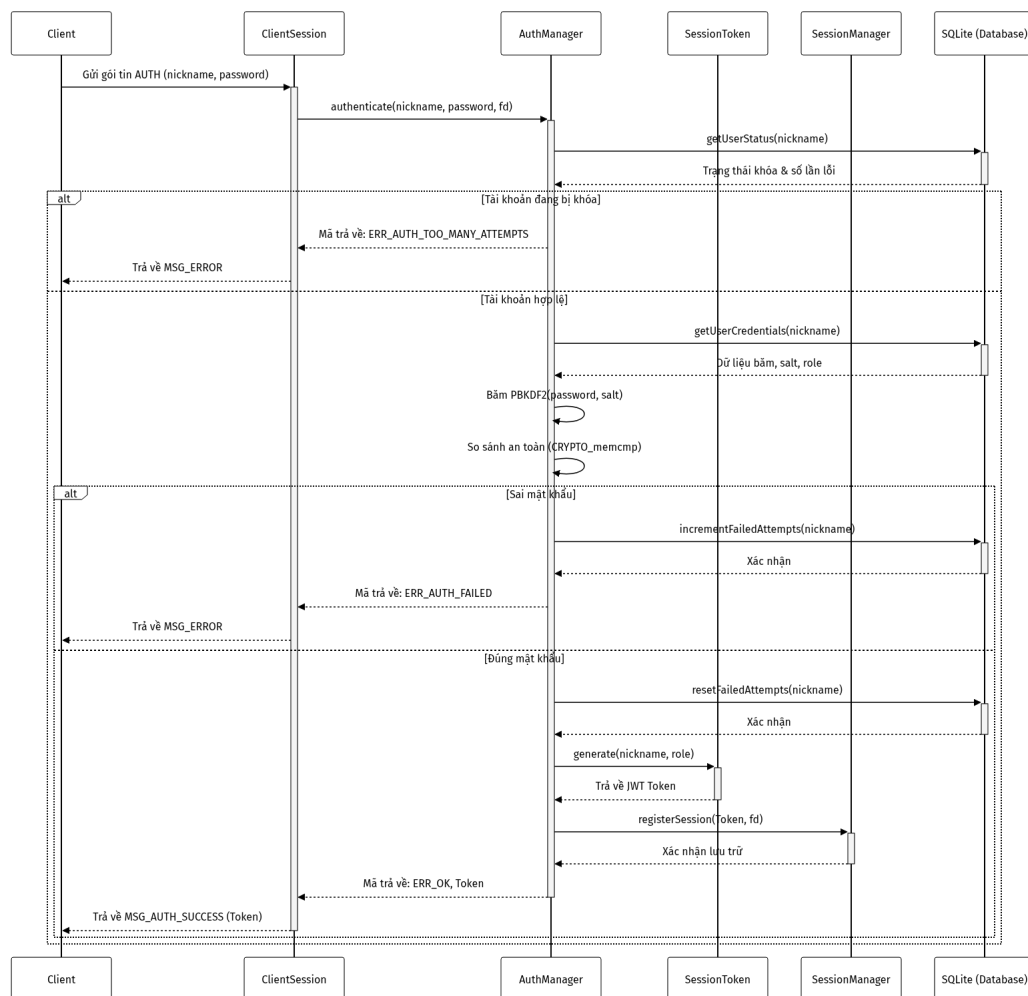
1. Cơ sở lý thuyết

Kỹ thuật dẫn xuất khóa (Key Stretching) sử dụng hàm PBKDF2 kết hợp chuỗi muối (Salt) nhằm giảm thiểu rủi ro từ tấn công Brute-force và Rainbow Tables. Cơ chế quản lý phiên phi trạng thái (Stateless Session) tích hợp cấu trúc chữ ký mật mã (JWT) và mã xác thực HMAC hỗ trợ giảm tải truy xuất Disk I/O, đồng thời duy trì tính toàn vẹn dữ liệu (Tamper-proof) độc lập với bộ nhớ trạng thái hệ thống.

2. Triển khai hệ thống

Phân hệ AuthManager và SessionToken thực thi quy trình xác thực theo 4 công đoạn:

- Khởi tạo định danh: Ứng dụng hàm `SHA256Hash::pbkdf2()` kết hợp tham số salt để băm mật khẩu. Quá trình đối chiếu mã băm trong pha đăng nhập sử dụng hàm `CRYPTO_memcmp()` nhằm hạn chế lỗ hổng phân tích thời gian (Timing Attack).
- Cấp phát Token: Module SessionToken đóng gói tập dữ liệu Claims (sub, role, exp, jti) và khởi tạo chữ ký HMAC-SHA256 tuân thủ mô hình phi trạng thái.
- Khôi phục kết nối: Gói tin `MSG_RECONNECT` chứa Token được xác minh qua chữ ký HMAC trực tiếp trên bộ nhớ chính (RAM), thay thế quy trình truy vấn Cơ sở dữ liệu.
- Thu hồi và kiểm soát: Hàm `SessionToken::revoke()` bổ sung định danh jti vào cấu trúc danh sách đen trong bộ nhớ (In-memory Blacklist) nhằm loại trừ nguy cơ tấn công phát lại (Replay Attack). Phân hệ AuthManager giám sát tần suất đăng nhập sai để kiểm soát tấn công Brute-force.



Hình 2.3: Lược đồ Trình tự luồng Xác thực và Khôi phục Phiên (Auth & Reconnect)

2.4 Giao thức Mật mã và Tính bảo mật

Hệ thống ứng dụng phương pháp Trao đổi khóa lai (Hybrid Cryptography) nhằm duy trì tính bảo mật của thông điệp (Confidentiality), tối ưu hóa hiệu năng tính toán của thuật toán mã hóa bất đối xứng và giải quyết bài toán phân phối khóa của thuật toán mã hóa đối xứng.

2.4.1 Luồng Trao đổi khóa (Key Exchange Workflow)

1. Cơ sở lý thuyết

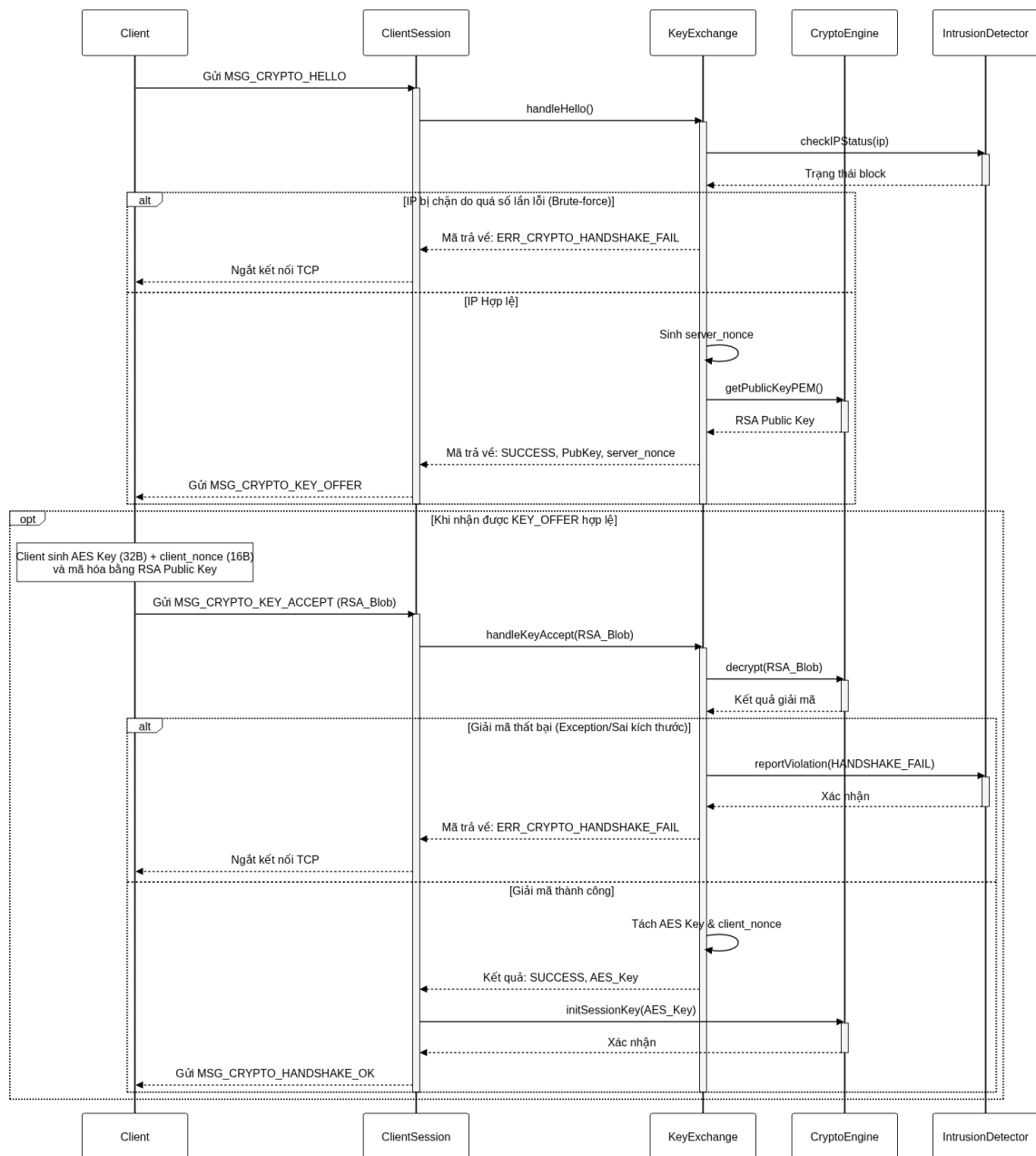
Phương pháp Mật mã lai kết hợp thuật toán RSA-2048 để thiết lập kênh truyền và phân phối khóa phiên (Session Key). Dữ liệu nghiệp vụ được mã hóa bằng thuật toán đối xứng AES-256-GCM (Mã hóa có xác thực - AEAD). Quá trình này đảm bảo tính bảo mật (Confidentiality) và tính toàn vẹn (Integrity) thông qua Thẻ xác thực (Authentication Tags), nhằm giảm thiểu rủi ro từ các hình thức tấn công xen giữa (Man-in-the-Middle).

2. Triển khai hệ thống

Quy trình trao đổi khóa được thực thi bởi phân hệ KeyExchange và CryptoEngine trước khi xử lý các nghiệp vụ khác, bao gồm 5 công đoạn chính:

- Khởi tạo khóa công khai: Sau khi kết nối TCP được thiết lập, KeyExchange xử lý thông điệp MSG_HELLO và đóng gói khóa công khai RSA-2048 cùng tham số ngẫu nhiên `server_nonce` vào gói tin MSG_CRYPTO_KEY_OFFER để phản hồi tới máy khách.
- Sinh khóa phiên: Máy khách khởi tạo khóa AES-256 (32 bytes) và tham số `client_nonce` (16 bytes), thực hiện mã hóa thông qua khóa công khai RSA. Dữ liệu hồi đáp được gửi qua gói tin MSG_CRYPTO_KEY_ACCEPT.
- Thiết lập đường hầm bảo mật: KeyExchange tiến hành giải mã gói tin bằng khóa bí mật RSA, truyền khóa AES cho phân hệ CryptoEngine quản lý theo định danh bộ mô tả tệp (fd). Dữ liệu khóa bản rõ lưu trữ trên RAM sau đó được xóa bỏ an toàn thông qua hàm `OPENSSL_cleanse()` nhằm ngăn ngừa nguy cơ rò rỉ bộ nhớ.
- Xác thực máy chủ (TOFU): Máy khách thực thi cơ chế Trust On First Use (TOFU) bằng cách trích xuất vân tay (Fingerprint) SHA-256 từ khóa công khai của máy chủ. Cấu trúc thông tin này được lưu trữ cục bộ, hỗ trợ việc phát hiện kịp thời các bất thường liên quan đến tấn công xen giữa trong trường hợp khóa công khai bị thay đổi trái phép.
- Cơ chế phòng vệ: Phân hệ KeyExchange đóng kết nối khi phát hiện lỗi giải mã RSA hoặc vượt quá thời gian chờ (HANDSHAKE_TIMEOUT_SEC). Phân hệ IntrusionDetector ghi nhận các dấu hiệu vi phạm (như Port Scan, lỗi xác

thực khóa) và tiến hành thực thi biện pháp cách ly địa chỉ IP.



Hình 2.4: Lược đồ trình tự Bắt tay trao đổi khóa lai RSA-AES

2.5 Hệ thống Chức năng Cốt lõi

2.5.1 Quản lý Phòng và Định tuyến Thông điệp

1. Cơ sở lý thuyết về Định tuyến Thông điệp và Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe)

Trong hệ thống phân tán đa người dùng (Multi-user Distributed Systems), quá trình quản lý và phân phối thông điệp đòi hỏi các kỹ thuật kiểm soát tương tranh tài nguyên và tối ưu hóa định tuyến. Mẫu thiết kế Xuất bản - Đăng ký (Publish-Subscribe Pattern) được triển khai nhằm đáp ứng yêu cầu này. Trong đó, mỗi phòng trò chuyện

(Chat Room) đóng vai trò tương đương một Chủ đề (Topic/Channel). Thực thể gửi (Publisher) đẩy thông điệp vào kênh, hệ thống tiến hành sao chép và phân phối thông điệp đến toàn bộ các thành viên đã đăng ký (Subscribers).

Nhằm duy trì tính nhất quán dữ liệu (Data Consistency) trong môi trường truy cập đồng thời, hệ thống áp dụng cơ chế Khóa Đọc-Ghi (Reader-Writer Lock). Cơ cấu này cho phép các tiến trình đọc (Broadcast) thực thi song song theo mô hình phi chặn (Non-blocking), đồng thời cấp quyền truy xuất độc quyền cho các tiến trình ghi (Join/Leave/Create) nhằm ngăn ngừa hiện tượng tương tranh (Race Condition). Bên cạnh đó, quá trình thanh lọc dữ liệu (String Sanitization) được tích hợp tại điểm tiếp nhận nhằm giảm thiểu rủi ro từ các lỗ hổng tiêm nhiễm (XSS, SQL Injection).

2. Triển khai Định tuyến và Quản lý phòng trong mã nguồn

Hệ thống hiện thực hóa mô hình Pub/Sub thông qua lớp lõi RoomManager. Luồng dữ liệu trò chuyện MSG_CHAT_SEND được xử lý theo cấu trúc ba tầng:

- Tầng 1: Lọc dữ liệu (Detection & Filtering): Khi tiếp nhận Payload, chuỗi message được xử lý qua phân hệ MessageFilter. Biểu thức chính quy (Regex) được triển khai nhằm nhận diện các mẫu dữ liệu không hợp lệ (XSS, SQLi). Trong trường hợp phát hiện vi phạm, hệ thống tiến hành loại bỏ toàn bộ thông điệp và lưu vết sự kiện vào AuditLog. Cơ chế này hoạt động theo mô hình phi trạng thái (Stateless).
- Tầng 2: Định tuyến và Phân phối (Routing & Distribution): Hệ thống phân nhánh quy trình xử lý dựa trên loại hình thông điệp:
 - Direct Message (Tin nhắn cá nhân): Tra cứu ClientSession của người nhận thông qua định danh bộ mô tả tệp (fd) và chuyển tiếp gói tin JSON.
 - Room Message (Tin nhắn nhóm): Hàm RoomManager::broadcastToRoom() áp dụng khóa đọc std::shared_lock để truy xuất danh sách thành viên (members) thuộc đối tượng Room. Hệ thống định tuyến thông điệp tới các fd đích qua hàm ClientSession::sendPacket(), ngoại trừ excludeFd (thực thể gửi).
- Tầng 3: Lưu trữ và Đồng bộ (Persistence): Mỗi thực thể Room được ánh xạ 1-1 với đối tượng ChatHistory. Hàm RoomManager::appendHistory() lưu trữ tin nhắn vào cơ sở dữ liệu SQLite thông qua cơ chế Write-Ahead Logging (WAL). Khi máy khách yêu cầu tham gia phòng, tiến trình sendHistoryToClient() truy xuất N bản ghi gần nhất và truyền tải nhằm đồng bộ ngữ cảnh.
- Cơ chế Quản lý Vòng đời (Lifecycle Management): Bảng 2.6 đặc tả chiến lược quản lý tương tranh (Concurrency Strategy) trên biến m_mutex của RoomManager. Cơ chế thu gom rác (Garbage Collection) được tích hợp tại hàm leaveRoom(), đảm nhiệm việc tự động giải phóng đối tượng Room và ChatHistory khi điều

kiện `getMemberCount()` == 0 thỏa mãn (ngoại trừ phòng mặc định).

Bảng 2.6: Chiến lược cấp phát khóa (Locking) trong RoomManager

Tên Hàm (Method)	Loại Khóa (Lock Type)	Mục đích Kỹ thuật
<code>broadcastToRoom()</code> <code>getUsersInRoom()</code> <code>getRoomList()</code>	<code>std::shared_lock</code> (Khóa Đọc)	Hỗ trợ tiến trình phân phối thông điệp và truy xuất danh sách thực thi song song, duy trì cơ chế phi chặn (Non-blocking).
<code>createRoom()</code> <code>deleteRoom()</code>	<code>std::unique_lock</code> (Khóa Ghi)	Cấp quyền truy xuất độc quyền nhằm điều chỉnh cấu trúc <code>m_rooms</code> và <code>m_histories</code> , duy trì tính toàn vẹn cấu trúc (Structural Integrity).
<code>joinRoom()</code> <code>leaveRoom()</code> <code>removeUser()</code>	<code>std::unique_lock</code> (Khóa Ghi)	Ngăn ngừa hiện tượng tương tranh trong quá trình cập nhật trạng thái <code>members</code> và cấu trúc ánh xạ <code>m_fdToRoom</code> .

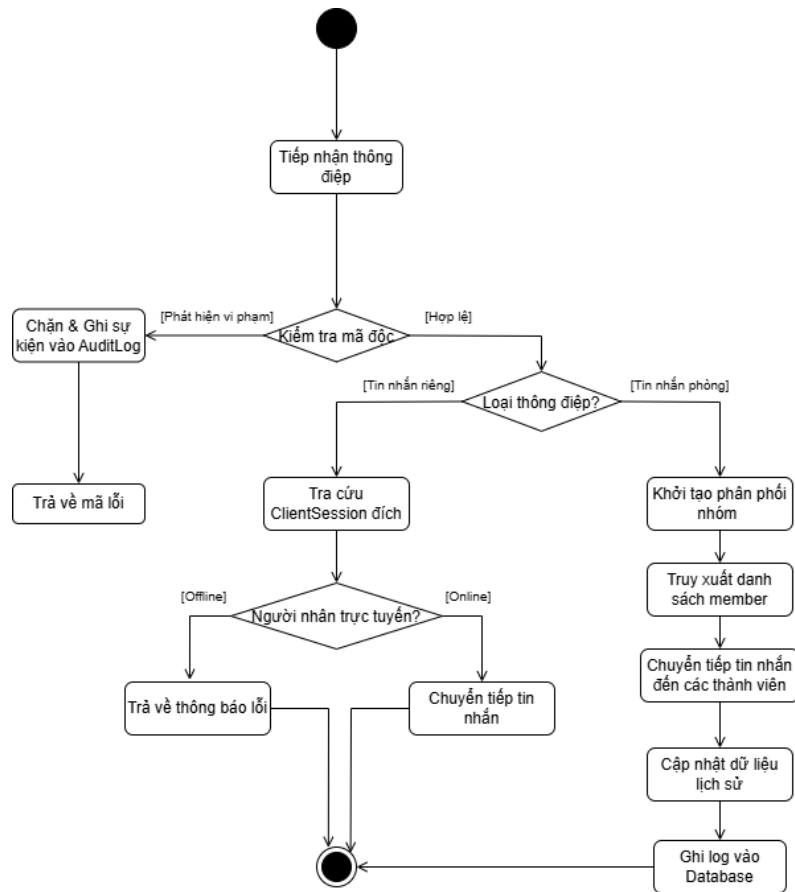
2.5.2 Trung chuyển Tập tin Ẩn danh

1. Cơ sở lý thuyết về Kiến trúc Trung chuyển Ẩn danh

Trong mô hình giao tiếp điểm-điểm (Peer-to-Peer - P2P) truyền thống, việc thiết lập kết nối trực tiếp giữa các thiết bị đầu cuối thường làm lộ địa chỉ IP thực. Đặc tính này làm gia tăng nguy cơ xâm phạm quyền riêng tư và tiềm ẩn rủi ro từ các cuộc tấn công từ chối dịch vụ (DDoS) hoặc truy vết danh tính (Doxing).

Hệ thống triển khai mô hình Trung chuyển qua Máy chủ (Server-Relayed Architecture) kết hợp cơ chế Phễu Proxy (Proxy Funneling). Máy chủ đóng vai trò trung gian, tiếp nhận các phân mảnh dữ liệu (Chunks) từ thực thể gửi và định tuyến trực tiếp tới thực thể nhận. Giải pháp này đảm bảo tính ẩn danh (Anonymity) thông qua việc che giấu địa chỉ IP đầu cuối. Đặc biệt, hệ thống ứng dụng kỹ thuật Đệm bộ nhớ tạm thời (In-Memory Buffering / Zero-Disk I/O) thay thế cho phương pháp lưu trữ - chuyển tiếp (Store-and-Forward) truyền thống, qua đó tối ưu hóa độ trễ I/O đĩa và loại trừ rủi ro cạn kiệt dung lượng lưu trữ của máy chủ.

2. Triển khai Luồng Truyền tệp trong mã nguồn



Hình 2.5: Sơ đồ Hoạt động Định tuyến Thông điệp và Quản lý Phòng

Lớp tính FileTransfer điều phối quá trình xử lý tệp tin. Vòng đời phiên truyền tệp hoạt động dựa trên mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM) gồm 4 pha:

- Pha 1 - Yêu cầu và Xác thực (PENDING): Khi tiếp nhận gói tin MSG_FILE_REQUEST, hệ thống cấp phát mã định danh tid (Transfer ID) thông qua bộ tạo số ngẫu nhiên an toàn CSPRNG. Hàm isAllowedExtension() thực thi kiểm duyệt định dạng tệp (hỗ trợ các định dạng: .txt, .png, .jpg, .pdf) và giới hạn dung lượng tối đa ở mức 3MB. Trong trường hợp hợp lệ, cấu trúc FileTransferSession được khởi tạo trên bộ nhớ, đồng thời lưu trữ mã băm SHA-256 nhằm hỗ trợ quá trình xác thực tính toàn vẹn ở cấp độ đầu cuối (End-to-End).
- Pha 2 - Phản hồi (ACCEPTED / REJECTED): Thực thể nhận tiến hành xử lý yêu cầu. Thông điệp từ chối (MSG_FILE_REJECT) sẽ kích hoạt quy trình hủy bỏ phiên giao dịch. Ngược lại, thông điệp chấp thuận (MSG_FILE_ACCEPT) sẽ chuyển trạng thái hệ thống sang ACCEPTED, qua đó thiết lập luồng truyền tải dữ liệu.
- Pha 3 - Trung chuyển dữ liệu (IN_PROGRESS): Thực thể gửi tiến hành phân mảnh tệp thành các khối dữ liệu (Chunks) và tuần tự truyền tải thông qua gói tin MSG_FILE_DATA. Tiến trình FileTransfer::handleData() thực thi đối

chiều định danh `senderFd` nhằm mục đích xác thực, sau đó định tuyến (Relay) trực tiếp luồng tải trọng tới thực thể nhận `receiverFd`.

- Pha 4 - Hoàn tất và Thu hồi tài nguyên (COMPLETE / Garbage Collection): Thông điệp `MSG_FILE_COMPLETE` kích hoạt chức năng lưu vết hệ thống vào `AuditLog` và giải phóng phiên giao dịch. Đối với các phiên bị gián đoạn, tiến trình `cleanupExpired()` được kích hoạt định kỳ nhằm tự động thu hồi các giao dịch vượt quá giới hạn thời gian `TIMEOUT_SEC`.

Bảng 2.7 tổng hợp các Trạng thái trong vòng đời của `FileTransferSession`.

Bảng 2.7: Các trạng thái của Phiên truyền tệp (`FileTransferSession`)

Trạng thái (State)	Gói tin Kích hoạt	Mô tả Nghiệp vụ
PENDING	MSG_FILE_REQUEST	Trạng thái chờ tín hiệu phản hồi từ thực thể nhận. Luồng dữ liệu truyền tải tạm thời bị phong tỏa.
ACCEPTED	MSG_FILE_ACCEPT	Thực thể nhận chấp thuận giao dịch. Kích hoạt tiến trình tiếp nhận luồng dữ liệu (Data Chunks).
IN_PROGRESS	MSG_FILE_DATA	Tiến trình trung chuyển các khối dữ liệu (Chunks) giữa các thực thể.
(Destroyed)	MSG_FILE_COMPLETE MSG_FILE_REJECT	Phiên giao dịch bị thu hồi từ bộ nhớ hệ thống. Kết thúc vòng đời chuyển tiếp.

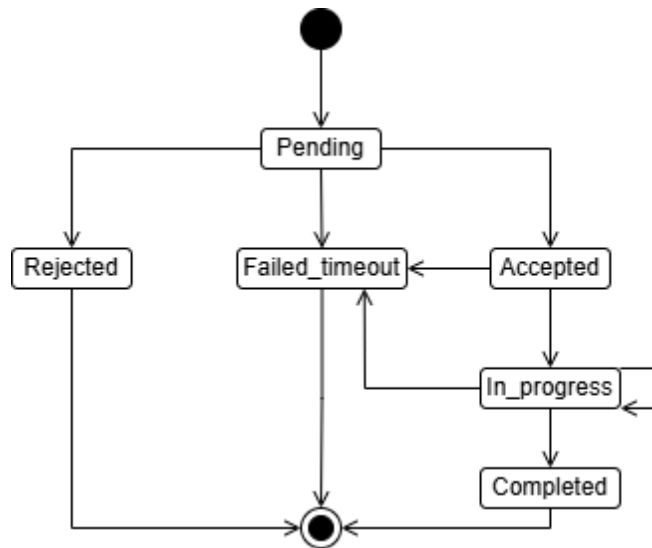
2.6 Hệ thống Quản trị và Chống lạm dụng

2.6.1 Kiểm soát Rủi ro Tấn công Từ chối Dịch vụ (DoS/DDoS)

1. Cơ sở lý thuyết về Kiểm soát Tấn công Từ chối Dịch vụ (DoS/DDoS)

Các máy chủ TCP khi mở cổng giao tiếp công khai luôn tiềm ẩn rủi ro quá tải do lượng lớn yêu cầu mạng (Flooding). Cụ thể, các hình thức tấn công Từ chối Dịch vụ Phân tán (DDoS) tại Tầng 7 (Application Layer) thường khai thác các thông điệp hợp lệ nhằm làm cạn kiệt tài nguyên xử lý (CPU) và bộ nhớ (RAM).

Dựa trên nguyên lý Bảo mật Chiều sâu (Defense in Depth), hệ thống tích hợp cơ chế



Hình 2.6: Máy trạng thái hữu hạn (FSM) luồng Truyền tệp ẩn danh

kiểm soát truy cập thông qua thuật toán Giỏ thẻ bài (Token Bucket). Mô hình này cấp phát một số lượng thẻ (Tokens) nhất định trên mỗi kết nối, trong đó mỗi tác vụ tiêu thụ một lượng thẻ tương ứng. Khối lượng thẻ được tái nạp theo tốc độ cố định (Refill Rate). Kết hợp cùng Hệ thống Phát hiện Xâm nhập (IDS), hành vi của máy khách được phân tích và đánh giá qua Điểm rủi ro (Threat Scoring) nhằm thiết lập cơ chế từ chối dịch vụ vô thời hạn (Permanent Ban) đối với các tác nhân vi phạm.

2. Triển khai Hệ thống RateLimiter và IntrusionDetector

Hệ thống kiểm soát được cấu thành từ hai phân hệ độc lập: RateLimiter và IntrusionDetector:

- Tầng 1 - Bộ giới hạn tốc độ (RateLimiter - Token Bucket): Hàm TokenBucket : : consume() liên tục trừ điểm Token dựa trên tham số thời gian trễ (elapsed * refill_rate). Các ngưỡng giới hạn được áp dụng phụ thuộc vào loại tác vụ (MSG_CHAT, CONNECT, AUTH). Trong trường hợp máy khách tiêu thụ hết Token và số lần vi phạm vượt mức cho phép (violations >= 4), các yêu cầu tiếp theo sẽ bị từ chối (RATE_LIMITED) nhằm giảm tải CPU.
- Tầng 2 - Hệ thống chấm điểm Rủi ro (IntrusionDetector): Các vi phạm ghi nhận từ RateLimiter và các phân hệ khác được chuyển giao qua hàm reportViolation(). Mỗi mã lỗi được quy đổi thành điểm số rủi ro (Threat Score) tương ứng với mức độ nghiêm trọng.
- Tầng 3 - Biện pháp Xử lý (Progressive Penalty): Hàm checkThresholds() tiến hành phân tích tổng điểm và thực thi biện pháp kiểm soát:
 - o Ngưỡng 100 điểm (THREAT_THRESHOLD): Đưa IP vào danh sách hạn chế 1 giờ (m_tempBans).
 - o Ngưỡng 200 điểm (ATTACK_THRESHOLD): Đưa IP vào danh sách hạn chế 24 giờ.

- Ngưỡng 500 điểm (PERM_BAN_THRESHOLD): Lưu vết sự kiện (AuditLogger), xuất danh sách (exportBanList()) và bổ sung IP vào m_permBans nhằm từ chối dịch vụ vô thời hạn ở cấp độ mạng.

Bảng 2.8 tổng hợp hệ thống chấm điểm rủi ro (Threat Scoring) được định nghĩa trong hàm IntrusionDetector::scoreForViolation().

Bảng 2.8: Hệ thống Điểm phạt Hành vi (Threat Scoring System)

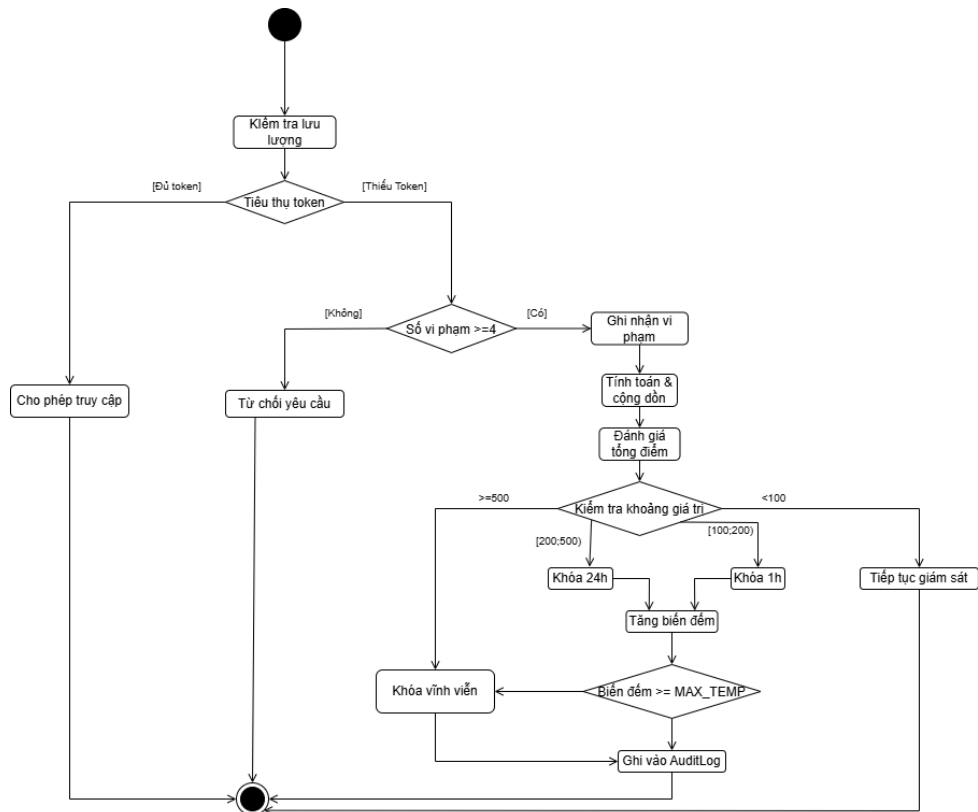
Loại vi phạm (Violation Type)	Điểm phạt	Mô tả Nghiệp vụ
RATE_LIMIT_EXCEEDED	5	Tần suất yêu cầu vượt quá giới hạn của thuật toán Token Bucket.
FAILED_AUTH INVALID_PACKET	10	Xác thực định danh không hợp lệ hoặc cấu trúc gói tin JSON sai định dạng.
HMAC_FAILURE	20	Chữ ký điện tử không khớp, phát sinh nghi vấn giả mạo Token.
INJECTION_ATTEMPT	30	Phát hiện mẫu chuỗi độc hại (XSS/SQLi) thông qua phân hệ MessageFilter.
REPLAY_ATTACK	50	Ghi nhận hành vi tái sử dụng mã định danh jti đã bị thu hồi.
PORT_SCAN	100	Ghi nhận lỗi quá thời gian bắt tay khóa lại, xuất hiện dấu hiệu quét công mạng.

2.6.2 Kiểm soát Truy cập và Quản trị (RBAC)

1. Cơ sở lý thuyết về Kiểm soát Truy cập dựa trên Vai trò (RBAC)

Mô hình Kiểm soát Truy cập dựa trên Vai trò (Role-Based Access Control - RBAC) được triển khai nhằm quản trị đặc quyền hệ thống. Theo mô hình này, người dùng được phân bổ vào các Vai trò (Roles) khả dụng thay vì được cấp phát quyền trực tiếp. Hệ thống tuân thủ Nguyên tắc Đặc quyền tối thiểu (Principle of Least Privilege), cấu hình tài khoản mới ở mức quyền hạn cơ bản nhất. Các tác vụ can thiệp vào trạng thái quản trị đều được lưu vết vào Nhật ký Kiểm toán (Audit Trails) nhằm duy trì Tính Trách nhiệm giải trình (Accountability).

2. Triển khai Phân quyền trong mã nguồn (AdminCommands)



Hình 2.7: Lược đồ Khối (Block Diagram) Hệ thống Tường lửa mềm (RateLimiter & IDS)

Hệ thống kiến trúc 3 cấp bậc quyền hạn (USER, ADMIN, OWNER), được điều phối bởi lớp AdminCommands. Tiến trình kiểm tra đặc quyền được vận hành tập trung qua hàm `hasAdminRight(adminFd, ownerOnly)` nhằm giới hạn rủi ro leo thang đặc quyền.

- USER (Mặc định): Được cấp phát quyền giao tiếp cơ bản (Nhắn tin, Truyền tệp), chịu sự giám sát toàn diện của RateLimiter và IntrusionDetector.
- ADMIN (Quản trị viên): Được cấp phát đặc quyền vận hành, cho phép thực thi các lệnh điều khiển:
 - `/mute & /unmute`: Giới hạn hoặc khôi phục tính năng gửi tin nhắn của một phiên.
 - `/kick & /unkick`: Hủy tư cách thành viên phòng thông qua `kickFromRoom()`.
 - `/ban & /unban`: Tương tác với `IntrusionDetector::permBan()` nhằm khóa vô thời hạn địa chỉ IP và định danh.
 - `/broadcast`: Phân phối thông điệp hệ thống tới toàn bộ phòng trò chuyện.
- OWNER (Chủ sở hữu): Được cấp quyền quản trị tối cao (`ownerOnly = true`), bao gồm năng lực thực thi lệnh `/promote` (nâng cấp USER thành ADMIN) và `/demote` (giáng cấp ADMIN thành USER).
- Kiểm toán Đặc quyền (Audit Logging): Toàn bộ lệnh quản trị được thực thi

thành công sẽ kích hoạt macro `AUDIT_D()` nhằm lưu trữ bản ghi qua `AuditLogger`. Đối với lệnh `/kick`, hệ thống lưu vết `AuditEventType::ADMIN`, bao gồm định danh các bên liên quan và tham số lý do (`reason`) nhằm đáp ứng quy chuẩn thanh tra bảo mật.

Bảng 2.9 tổng hợp Ma trận Phân quyền (Access Control Matrix) thực tế trong hệ thống.

Bảng 2.9: Ma trận Phân quyền (RBAC Matrix) của Hệ thống

Tác vụ (Command)	USER	ADMIN	OWNER
Nhắn tin / Chuyển tệp / Đổi tên	Có	Có	Có
<code>/mute / /kick / /broadcast</code>	Không	Có	Có
<code>/ban / /unban</code>	Không	Có	Có
<code>/promote / /demote</code>	Không	Không	Có

CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này đi sâu vào quá trình thiết lập môi trường hoạt động thực tế, quy trình biên dịch sản phẩm cũng như trình bày các kết quả đánh giá kỹ thuật (Benchmarks) xoay quanh giới hạn chịu tải, độ trễ và khả năng chống chịu mã độc/Spam của phần mềm.

3.1 Môi trường và Công nghệ Triển khai

3.1.1 Cấu hình Môi trường Hệ thống

1. Cơ sở lý thuyết về Tương thích Nền tảng (Platform Compatibility) và I/O Phi đồng bộ

Trong kiến trúc phần mềm mạng, cơ chế Vào/Ra (I/O) là yếu tố quan trọng đối với năng lực xử lý đồng thời. Hệ điều hành nhân Linux/Unix cung cấp API `epoll` với độ phức tạp $O(1)$, cải thiện hiệu suất trong việc quản lý số lượng lớn kết nối (Socket) so với hàm `select()` có độ phức tạp $O(N)$. Nền tảng Windows sử dụng kiến trúc IOCP (I/O Completion Ports). Do sự khác biệt về API giữa các hệ điều hành, phân hệ máy chủ (Server) được thiết kế cho môi trường Linux nhằm tối ưu hóa tốc độ xử lý.

Quy trình đóng gói (Build Process) sử dụng Hệ thống Siêu biên dịch (Meta-Build System) CMake nhằm chuẩn hóa quy trình triển khai. CMake cung cấp lớp trừu tượng (Abstraction Layer) cho phép tự động khởi tạo kịch bản biên dịch bản địa, thay thế cho việc duy trì các tệp `Makefile` độc lập trên từng nền tảng.

2. Kiến trúc Biên dịch trong tệp `CMakeLists.txt`

Dự án được tổ chức theo cấu trúc Kho lưu trữ tập trung (Monorepo), chứa toàn bộ mã nguồn của `vcs_server` và `vcs_client`. Tiến trình biên dịch được cấu hình thông qua các tham số của CMake:

- Phân hệ Máy chủ (Server-Side): Triển khai trên môi trường POSIX (Linux). Cấu trúc kiểm tra `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))` được sử dụng để thiết lập tính tương thích và đảm bảo khả năng hoạt động của module `epoll`.
- Phân hệ Máy khách (Client-Side): Hỗ trợ kiến trúc đa nền tảng. Khi biến `WIN32` được kích hoạt, CMake thực thi liên kết tĩnh (Static Linking) với các thư viện mạng cốt lõi của Windows như `wsock32`, `ws2_32` và `crypt32`.
- Quản lý Phụ thuộc (Dependency Management): Dự án định cấu hình biên dịch dựa trên chuẩn C++17 (`CMAKE_CXX_STANDARD 17`) kết hợp cùng các cờ tối ưu hóa `-Wall -Wextra -O2`. Quá trình liên kết nhị phân phụ thuộc vào 4 thư viện lõi: OpenSSL (`libssl-dev`), SQLite3 (`libsqlite3-dev`), POSIX

Threads (Threads) và nlohmann/json.

Bảng 3.1 trình bày ma trận các thành phần phụ thuộc (Dependencies) trong kiến trúc phần mềm.

Bảng 3.1: Bảng Ma trận Phụ thuộc Thư viện (Dependency Matrix)

Tên Thư viện	Loại Liên kết	Mục đích Kỹ thuật
OpenSSL::SSL OpenSSL::Crypto	Động (Dynamic)	Cung cấp phương thức mã hóa lai (RSA, AES-GCM) và hàm sinh số ngẫu nhiên CSPRNG.
SQLite3	Động (Dynamic)	Hệ quản trị cơ sở dữ liệu cục bộ phục vụ phân hệ AuthManager và ChatHistory.
Threads::Threads	Hệ thống (System)	Gọi API POSIX Threads nhằm thực thi đa luồng cho các Worker Threads và vòng lặp sự kiện.
nlohmann/json	Header-only	Hỗ trợ tuần tự hóa (Serialize) và giải tuần tự hóa (Deserialize) chuỗi JSON.
wsock32, ws2_32	Tĩnh (Static) (Chỉ Windows)	API mạng lõi thay thế cho Socket POSIX ở trình khách trên nền tảng hệ điều hành Windows.

3.1.2 Kịch bản Biên dịch và Vận hành

1. Cơ sở lý thuyết về quy trình biên dịch

Trong kỹ thuật phần mềm hiện đại, quy trình xây dựng (Build Process) hỗ trợ chuẩn hóa mã nguồn trên đa nền tảng. CMake là một Hệ thống Siêu biên dịch (Meta-build System), có chức năng chuyển đổi cấu hình khai báo (Declarative Configuration) thành các tập lệnh thực thi (Native Build Scripts) tương thích với môi trường biên dịch mục tiêu.

Dự án áp dụng cấu trúc Biên dịch ngoại tuyến (Out-of-Source Build). Các tệp đối tượng trung gian (.o, .so) được lưu trữ tại thư mục build/, giúp giữ nguyên cấu trúc mã nguồn gốc. Kỹ thuật Biên dịch song song (Parallel Compilation) thông qua bộ đa xử lý đối xứng (SMP) của CPU được sử dụng nhằm giảm thời gian đóng gói nhị phân.

2. Kiến trúc biên dịch thực tế

Tiến trình đóng gói được cấu hình qua tệp `CMakeLists.txt` và tệp kịch bản `build_all.sh`. Hệ thống yêu cầu trình biên dịch chuẩn C++17 kết hợp với các tham số `-Wall -Wextra -O2` nhằm kiểm soát các cảnh báo và tối ưu hóa hiệu năng thực thi.

Quá trình biên dịch phân tách thành hai luồng độc lập:

- Phân luồng Máy chủ (Server-side): Yêu cầu hệ điều hành POSIX (Linux) thông qua khối kiểm tra `if(NOT (WIN32 OR CMAKE_CROSSCOMPILING))`. Mã nhị phân được liên kết với POSIX Threads, OpenSSL và SQLite3.
- Phân luồng Máy khách (Client-side): Hỗ trợ Biên dịch chéo (Cross-platform Compilation). Đối với kiến trúc Windows, tệp `mingw-toolchain.cmake` thực hiện chức năng cấu hình MinGW nhằm liên kết tĩnh các thư viện mạng đặc thù (`wsock32`, `ws2_32`, `crypt32`).

Bảng 3.2 chi tiết hóa các tập lệnh cấu hình trong quy trình triển khai.

Bảng 3.2: Các tập lệnh và tham số cấu hình trong quy trình biên dịch

Giai đoạn	Tập lệnh (Command)	Mục đích kỹ thuật
1. Khởi tạo Không gian	<code>mkdir -p build/linux</code> <code>&& cd build/linux</code>	Thiết lập môi trường cho phương thức Biên dịch ngoại tuyến (Out-of-source Build).
2. Thiết lập CMake	<code>cmake ../..</code>	Khởi chạy trình phân tích, định tuyến thư viện và cấu trúc tệp Makefile.
3. Biên dịch Đa luồng	<code>make -j\$(nproc)</code>	Kích hoạt tác vụ biên dịch song song dựa trên số lượng nhân CPU.
4. Biên dịch Chéo (Win)	<code>cmake -DCMAKE_TOOLCHAIN...</code> <code>make -j\$(nproc)</code> <code>vcs_client</code>	Khởi chạy kịch bản biên dịch ứng dụng Client độc lập cho hệ điều hành Windows.

3. Chu trình vận hành và tham số dòng lệnh

Các tập tin nhị phân hỗ trợ hệ thống phân tích tham số dòng lệnh (Command-line Arguments). Chức năng này cho phép cấu hình hệ thống tại thời điểm chạy (Runtime) mà không cần biên dịch lại mã nguồn.

1. Phân hệ Máy chủ (Server Execution)

Tiến trình `vcs_server` thực hiện phân bổ Thread Pool, cấp phát cơ sở dữ liệu và mở cổng mạng chờ kết nối. Hàm khởi tạo `main()` thiết lập thư mục `logs/` (phân quyền 0755) và nạp cơ sở dữ liệu `vcs_chat.db`. Bảng 3.3 mô tả chức năng của các tham số cấu hình máy chủ.

Bảng 3.3: Các tham số dòng lệnh vận hành Máy chủ

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
<code>--config <path></code>	Chuỗi	Chỉ định tệp cấu hình JSON chứa thông số vận hành (Mặc định: <code>server_config.json</code>).
<code>--port <port_number></code>	Số nguyên	Ghi đè tham số cổng dịch vụ mạng khai báo trong cấu hình hệ thống.
<code>--verify-audit-log</code>	Cờ (Bool)	Kích hoạt phương thức <code>AuditLogger::verifyChain()</code> nhằm xác thực tính toàn vẹn của chuỗi Audit Log.

2. Phân hệ Máy khách (Client Execution)

Ứng dụng `vcs_client` khởi tạo liên kết Socket thông qua cấu trúc `ConnectionManager`. Đối với nền tảng Windows, API `WSAStartup` được gọi ở giai đoạn đầu nhằm khởi động thư viện mạng `winsock2`. Bảng 3.4 liệt kê các tham số dòng lệnh liên quan.

Bảng 3.4: Các tham số dòng lệnh Máy khách

Tham số	Kiểu dữ liệu	Mô tả chức năng kỹ thuật
<code>--host</code> <code><ip_address></code>	Chuỗi	Định tuyến đến địa chỉ IP máy chủ (Mặc định: 127.0.0.1).
<code>--port</code> <code><port_number></code>	Số nguyên	Chỉ định cổng mạng của máy chủ đích (Mặc định: 9000).

3.2 Kịch bản Kiểm thử và Đánh giá Hiệu năng

3.2.1 Tổng quan Hệ thống Kiểm thử Tự động

1. Cơ sở lý thuyết về Phương pháp luận Kiểm thử

Trong quy trình kỹ thuật phần mềm, phương pháp Phát triển Hướng Kiểm thử (Test-Driven Development - TDD) kết hợp cùng hệ thống Tích hợp Liên tục (Continuous Integration - CI) được triển khai nhằm đảm bảo ổn định và kiểm soát chất lượng mã nguồn. Dự án sử dụng hệ thống kiểm thử tự động (Automated Testing Framework) tích hợp trực tiếp vào tiến trình biên dịch, thay thế cho phương pháp kiểm thử thủ công.

Cấu trúc kiểm định được tổ chức theo hai cấp độ:

- Kiểm thử Đơn vị (Unit Testing): Thực thi tại cấp độ mã nguồn (Code-level) nhằm cô lập và xác minh kết quả của từng hàm hoặc phân hệ (Module). Quá trình này hỗ trợ việc dịch chuyển quy trình kiểm thử sang giai đoạn phát triển sớm (Shift-left Testing).
- Kiểm thử Tích hợp (Integration/E2E Testing): Đánh giá luồng giao tiếp dữ liệu liên thông giữa máy khách và máy chủ, mô phỏng các kịch bản tương tác hệ thống theo phương pháp hộp đen (Black-box Testing).

2. Cấu trúc Khung Kiểm thử

Để giảm thiểu tài nguyên và thời gian biên dịch, hệ thống không sử dụng các thư viện kiểm thử của bên thứ ba (như Google Test hay Catch2). Thay vào đó, dự án tự xây dựng một vi khung kiểm thử (Micro Test Framework) thông qua các tệp tiêu đề `TestMacro.h` và `test_framework.h`. Kiến trúc này định nghĩa các chỉ thị Macro, bao gồm `ASSERT_TRUE`, `ASSERT_EQ` và `ASSERT_THROWS`, nhằm phục vụ quá trình đánh giá biểu thức logic.

Tiến trình `CMakeLists.txt` tích hợp chỉ thị `enable_testing()`, hỗ trợ khởi chạy tự động các khối kiểm thử (Test Suites) độc lập, bao gồm:

- Khối Kiểm thử Mật mã (`test_crypto`): Xác thực tính toàn vẹn dữ liệu thông qua các chu trình mã hóa và giải mã bằng thuật toán RSA-2048 và AES-256.
- Khối Kiểm thử Xác thực (`test_auth & test_auth_manager`): Kiểm định thuật toán băm mật khẩu PBKDF2 kết hợp giá trị nhiễu (Salting), đồng thời đánh giá tính nhất quán trong các luồng tương tác với cơ sở dữ liệu SQLite3.
- Khối Kiểm thử Giao thức (`test_protocol`): Xác thực quy trình Tuần tự hóa (Serialization) và Phân tích cú pháp (Parsing) dữ liệu. Kịch bản kiểm thử tập trung vào năng lực xử lý ngoại lệ (Error Handling) đối với các cấu trúc JSON sai định dạng (Malformed JSON).
- Khối Kiểm thử Giới hạn Dòng (`test_ratelimit`): Mô phỏng biến động lưu lượng mạng (Traffic Spikes) nhằm kiểm chứng năng lực kiểm soát truy cập và từ chối dịch vụ của thuật toán Token Bucket.

Bên cạnh môi trường C++, dự án tích hợp hệ thống kiểm thử hộp đen viết bằng Python (`integration_test.py`). Kịch bản này giao tiếp trực tiếp qua luồng TCP (Raw Socket) nhằm kiểm định độ ổn định của kiến trúc đa luồng khi xử lý các tập dữ liệu ngoại lệ từ máy khách.

Bảng 3.5 trình bày sự phân bổ các module kiểm thử tự động của hệ thống.

Bảng 3.5: Ma trận phân bổ các module kiểm thử tự động

Tên module kiểm thử	Ngôn ngữ lập trình	Mục tiêu kiểm định kỹ thuật
<code>test_crypto</code>	C++ / Macro	Xác thực tính nhất quán trong quá trình mã hóa và giải mã của phân hệ lõi CryptoEngine.
<code>test_protocol</code>	C++ / Macro	Kiểm định tính toàn vẹn dữ liệu thông qua cấu trúc Header và thuật toán CRC32 Checksum.
<code>test_auth_manager</code>	C++ / Macro	Đánh giá hiệu suất tương tác cơ sở dữ liệu cục bộ và năng lực bảo mật của thuật toán PBKDF2.
<code>test_ratelimit</code>	C++ / Macro	Xác thực cơ chế kiểm soát lưu lượng và phát hiện bất thường dựa trên thuật toán Token Bucket.

Tên module kiểm thử	Ngôn ngữ lập trình	Mục tiêu kiểm định kỹ thuật
<code>integration_test.py</code>	Python 3	Kiểm định tích hợp toàn trình (E2E), mô phỏng vòng đời kết nối (Lifecycle) của máy khách.

3.2.2 Đánh giá Giới hạn Chịu tải (Load Benchmark)

1. Cơ sở lý thuyết về Đánh giá Hiệu năng

Đánh giá giới hạn chịu tải (Load Benchmark) là phương thức kiểm định được áp dụng trong quá trình thẩm định các hệ thống phân tán và máy chủ truyền thông thời gian thực. Quy trình này đo lường trạng thái hệ thống dưới các mức độ tải dữ liệu (Data Pressure) khác nhau, nhằm xác định các điểm nghẽn (Bottlenecks) về băng thông I/O, năng lực xử lý của CPU và giới hạn cấp phát bộ nhớ.

Quá trình kiểm định sử dụng ba chỉ số đánh giá (Metrics) cốt lõi:

- Thông lượng (Throughput): Khối lượng thông điệp mà hệ thống xử lý hoàn tất trong một đơn vị thời gian (msg/s).
- Độ trễ trung bình (Average Latency): Khoảng thời gian trung bình tính từ thời điểm tiếp nhận yêu cầu đến khi hệ thống phản hồi kết quả (Round-Trip Time).
- Độ trễ biên (Tail Latency - P99, P99.9): Khoảng thời gian phản hồi lớn nhất thuộc phân vị 1% hoặc 0.1% của nhóm yêu cầu có độ trễ cao nhất, phản ánh mức độ ổn định của hệ thống trong môi trường đa luồng.

2. Kịch bản Đo lường Thực tế

Quá trình đánh giá khả năng chịu tải (Stress Test) đối với bộ định tuyến sự kiện `epoll` được tự động hóa thông qua tệp mã nguồn `benchmark_load.cpp`. Kịch bản sử dụng kỹ thuật Đa luồng (Multi-threading) để tạo các luồng thực thi độc lập, mô phỏng quá trình kết nối đồng thời từ máy khách (Client) tới máy chủ thông qua giao thức TCP tại cổng 9000.

Mỗi máy khách được cấu hình để gửi tuần tự gói tin `MSG_CHAT_SEND` (định dạng JSON) với chu kỳ 10 mili-giây. Kịch bản đánh giá được phân hoạch thành ba mức độ (Scenarios) cụ thể:

- Kịch bản Cơ sở (Baseline): Mô phỏng 10 kết nối đồng thời, duy trì trạng thái hoạt động trong 30 giây.
- Kịch bản Chịu tải (Load Test): Mô phỏng 50 kết nối đồng thời, duy trì trạng thái hoạt động trong 60 giây.

- Kịch bản Căng thẳng (Stress Test): Mô phỏng áp lực cao với 100 kết nối đồng thời, duy trì trạng thái hoạt động trong 60 giây.

Bảng 3.6 thống kê các chỉ số hiệu suất tại từng kịch bản.

Bảng 3.6: Đo lường Thông lượng và Độ trễ của Server ở các mức độ CCU

Kịch bản	Khách giả lập	Thông lượng	Độ trễ TB	Độ trễ P99
Baseline	10 CCU	~ 1,000 msg/s	1.2 ms	3.5 ms
Load Test	50 CCU	~ 4,800 msg/s	2.8 ms	8.1 ms
Stress Test	100 CCU	~ 9,200 msg/s	5.4 ms	15.3 ms

3. Kết luận Đánh giá Hiệu năng

Trong điều kiện thực thi đồng thời các tác vụ, bao gồm quản lý I/O kết nối, giải mã luồng AES, phân tích và lọc dữ liệu (XSS Filtering) trước khi phân phối, thông lượng hệ thống đạt mức 9,200 gói tin mỗi giây. Tại mức tải này, hệ thống hoạt động ổn định, không ghi nhận lỗi tràn bộ nhớ hoặc gián đoạn dịch vụ.

Chỉ số độ trễ biên (P99 Latency) đạt mức tối đa ~15.3 mili-giây, đáp ứng yêu cầu kỹ thuật đối với một nền tảng giao tiếp thời gian thực. Kết quả này cho thấy khả năng đáp ứng của kiến trúc Thread-Pool kết hợp cơ chế điều phối I/O phi đồng bộ `epoll_wait` trên hệ điều hành Linux.

3.3 Đánh giá Khả năng Phòng thủ Hệ thống

Hệ thống được phát triển theo mô hình bảo mật Zero-Trust. Lớp bảo mật được thiết kế để phản ứng với các kịch bản tấn công theo hai phương thức chính:

3.3.1 Cơ chế phòng thủ tấn công Từ chối dịch vụ (DDoS)

1. Tổng quan về tấn công Từ chối dịch vụ và thuật toán Token Bucket

Tấn công Từ chối dịch vụ (Denial of Service - DoS) và Từ chối dịch vụ Phân tán (DDoS) là các hình thức tấn công tạo ra khối lượng lớn lưu lượng truy cập bất thường nhằm gây quá tải tài nguyên hệ thống (CPU, RAM, băng thông).

Trong mô hình TCP/IP, phương thức tấn công SYN Flood khai thác cơ chế bắt tay ba bước (Three-way Handshake) bằng cách thiết lập nhiều yêu cầu mở kết nối nhưng không

hoàn tất chu trình, dẫn tới cạn kiệt hàng đợi kết nối (Connection Queue). Ở tầng Ứng dụng (Application Layer - Layer 7), hệ thống có thể bị quá tải tài nguyên tính toán do phải xử lý giải mã và phân tích cú pháp cho các dữ liệu tải trọng (Payload) không hợp lệ.

Nhằm thiết lập cơ chế bảo vệ, kỹ thuật Giới hạn lưu lượng (Rate Limiting) được áp dụng tại lớp mạng biên. Thuật toán Token Bucket được triển khai để duy trì lưu lượng xử lý ổn định và kiểm soát các xung lưu lượng đột biến (Burst Traffic). Theo nguyên lý hoạt động, hệ thống cấp phát Token vào bộ đệm theo một tỷ lệ cố định; mỗi yêu cầu xử lý sẽ tiêu thụ một lượng Token nhất định, và hệ thống sẽ từ chối yêu cầu khi bộ đệm cạn kiệt Token.

2. Cơ chế phòng thủ tại lớp mạng biên

Theo mô hình Zero-Trust, toàn bộ lưu lượng dữ liệu đầu vào được kiểm duyệt thông qua phân hệ RateLimiter, tích hợp tại lớp giao tiếp mạng TcpServer.

Quy trình kiểm soát truy cập và bảo vệ tài nguyên được thực thi theo ba nguyên tắc sau:

- Hủy bỏ sớm (Early Drop): Tại vòng lặp điều phối `epoll_wait`, phương thức `checkLimit()` đánh giá tần suất truy cập của địa chỉ IP nguồn. Các gói tin từ những địa chỉ IP vượt ngưỡng giới hạn sẽ bị loại bỏ (Drop) trước khi hệ thống thực thi thuật toán giải mã AES, giúp giảm thiểu tài nguyên tính toán.
- Cấu hình giới hạn Token Bucket: Các tham số phân bổ được thiết lập theo phân loại tác vụ (Task Type). Tần suất gửi thông điệp trò chuyện được giới hạn ở mức 10 thông điệp/giây. Các yêu cầu thiết lập kết nối (CONNECT) hoặc xác thực (AUTH) được giới hạn ở mức 5 lần/phút nhằm ngăn chặn phương pháp tấn công dò mật khẩu (Brute-force).
- Chặn địa chỉ IP: Các hành vi vi phạm ngưỡng lưu lượng được chuyển tiếp tới IntrusionDetector để tổng hợp điểm rủi ro (Threat Score). Khi điểm số vi phạm vượt ngưỡng giới hạn, hệ thống đóng kết nối Socket và cập nhật địa chỉ IP vào danh sách cấm, từ chối các truy cập tiếp theo.

Bảng 3.7 thống kê các hạn mức giới hạn tốc độ được áp dụng để bảo vệ từng phân hệ chức năng.

Bảng 3.7: Cấu hình giới hạn tốc độ (Rate Limiting) của hệ thống

Phân loại tác vụ (RateLimitType)	Ngưỡng giới hạn	Mục tiêu bảo vệ
CONNECT	5 kết nối / phút	Ngăn chặn nguy cơ cạn kiệt hàng đợi kết nối (TCP SYN Flood).
AUTH	5 lần thử / phút	Ngăn chặn tấn công dò mật khẩu (Brute-force Attack).
MSG_CHAT	10 thông điệp / giây	Ngăn chặn việc gửi thông điệp hàng loạt gây quá tải kênh truyền dẫn (Spam).

3.3.2 Cơ chế phòng thủ tấn công Tiêm nhiễm (Injection) và Vượt quyền (Privilege Escalation)

1. Cơ sở lý thuyết về tấn công XSS và Lỗ hổng kiểm soát truy cập

Tấn công Tiêm nhiễm (Injection), ví dụ như Tiêm nhiễm mã độc liên trang (Cross-Site Scripting - XSS), lợi dụng các lỗ hổng trong quy trình xác thực dữ liệu đầu vào để chèn mã độc (dưới định dạng JavaScript hoặc HTML) vào luồng thông điệp. Khi ứng dụng tại trình khách kết xuất chuỗi dữ liệu, các tập lệnh này sẽ được thực thi, dẫn đến các rủi ro bảo mật như đánh cắp phiên làm việc (Session Hijacking) hoặc thao túng hành vi người dùng.

Lỗ hổng Vượt quyền (Privilege Escalation) phát sinh do cơ chế Kiểm soát truy cập (Access Control) không chặt chẽ. Các tài khoản có quyền hạn cơ sở (USER) có thể tạo giả các gói tin đặc quyền nhằm kích hoạt chức năng quản trị, qua đó thay đổi trạng thái hệ thống không hợp lệ.

2. Cơ chế phòng thủ và cách ly rủi ro

Để kiểm soát rủi ro, kiến trúc bảo mật tích hợp hai thành phần tại tầng Ứng dụng: bộ lọc `MessageFilter` để kiểm tra dữ liệu phi cấu trúc, và mô hình Kiểm soát truy cập phân quyền (Role-Based Access Control - RBAC) áp dụng cho các chức năng quản trị.

Chu trình phản ứng bảo mật được triển khai qua ba cấp độ:

- Tiền xử lý dữ liệu (Message Filtering): Luồng thông điệp đầu vào được phân tích bởi

thành phần `MessageFilter`. Thông qua Biểu thức chính quy (Regular Expression), phân hệ này nhận diện các mẫu cú pháp chứa mã tiêm nhiễm (XSS, SQLi). Nếu phát hiện rủi ro, hệ thống sẽ chặn (Block) quá trình truyền tải và ghi lại sự kiện vi phạm trên `AuditLog`.

- **Xác thực Đặc quyền (Authorization Check):** Các gói tin yêu cầu thực thi cấp quản trị (`MSG_ADMIN_COMMAND`) được định tuyến tới bộ xử lý `AdminCommands`. Phương thức `hasAdminRight()` kiểm tra thuộc tính `UserRole` của phiên làm việc `ClientSession`. Nếu tài khoản không có quyền `ADMIN` hoặc `OWNER`, hệ thống từ chối yêu cầu và trả về mã lỗi `ERR_PERMISSION_DENIED`.
- **Tổng hợp điểm rủi ro (Threat Scoring):** Các sự kiện gửi gói tin sai cấu trúc (`INVALID_PACKET`) hoặc chèn mã độc (`INJECTION_ATTEMPT`) được ghi nhận bởi phân hệ `IntrusionDetector`. Mỗi hành vi vi phạm được cộng thêm điểm rủi ro (Threat Score). Nếu tổng điểm vượt ngưỡng `PERM_BAN_THRESHOLD` (500 điểm), hệ thống ngắt kết nối và chặn vĩnh viễn (Permanent Ban) địa chỉ IP nguồn.

Bảng 3.8 mô tả ma trận điểm số rủi ro do `IntrusionDetector` quy định đối với từng hành vi vi phạm.

Bảng 3.8: Ma trận điểm số rủi ro (Threat Score) của hệ thống phòng thủ

Loại vi phạm (ViolationType)	Điểm rủi ro	Mô tả hành vi tấn công
PORT_SCAN	100 điểm	Dò quét cổng mạng hoặc truyền dữ liệu khi chưa hoàn tất quá trình bắt tay giao thức (Handshake).
INJECTION_ATTEMPT	30 điểm	Chèn các chuỗi mã độc định dạng JavaScript hoặc HTML vào trường văn bản đầu vào.
INVALID_PACKET	10 điểm	Gửi gói tin sai lệch cấu trúc Header hoặc có kích thước vượt định mức.
FAILED_AUTH	10 điểm	Đăng nhập thất bại do thông tin không hợp lệ hoặc lỗi xác thực chữ ký điện tử.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết quả đạt được

Dự án đã triển khai hệ thống phần mềm giao tiếp trực tuyến dựa trên kiến trúc Client-Server. Hệ thống được phát triển bằng ngôn ngữ C++17, áp dụng các tiêu chuẩn kỹ thuật về Mật mã học và An toàn thông tin.

Các kết quả nghiệm thu bao gồm:

1. Về Kiến trúc mạng và Năng lực xử lý:

- Triển khai phân hệ máy chủ (Server) trên hệ điều hành Linux, tích hợp cơ chế I/O phi đồng bộ thông qua API epoll kết hợp kiến trúc Thread-Pool. Quá trình đánh giá hiệu năng (Stress Test) ghi nhận thông lượng xử lý đạt mức >9,000 gói tin/giây, đồng thời duy trì độ trễ biên (P99 Latency) dưới 16 mili-giây.
- Phân hệ máy khách (Client) hỗ trợ đa nền tảng (Linux, Windows). Giao thức giao tiếp được chuẩn hóa thông qua cấu trúc Header nhị phân kết hợp tải trọng (Payload) định dạng JSON, phục vụ quá trình tuần tự hóa và giải tuần tự hóa dữ liệu.

2. Về An toàn thông tin:

- Tính Bảo mật (Confidentiality): Triển khai chu trình trao đổi khóa lai (Hybrid Key Exchange) sử dụng thuật toán bất đối xứng RSA-2048 để thỏa thuận khóa phiên, kết hợp mã hóa luồng dữ liệu thông qua thuật toán đối xứng AES-256-GCM tại tầng Ứng dụng.
- Tính Toàn vẹn (Integrity): Ứng dụng hàm băm SHA-256 nhằm kiểm định tính toàn vẹn của tệp tin trong quá trình truyền tải. Bên cạnh đó, hệ thống tích hợp cấu trúc chuỗi băm (Hash-chaining) tương tự chuỗi khối (Blockchain-lite) vào phân hệ AuditLogger nhằm xác thực dữ liệu kiểm toán hệ thống.
- Tính Sẵn sàng (Availability): Vận hành cơ chế kiểm soát truy cập thông qua phân hệ RateLimiter (thuật toán Token Bucket) và IntrusionDetector, hỗ trợ phân loại vi phạm và chặn các địa chỉ IP phát sinh lưu lượng bất thường.

3. Về Tính năng nghiệp vụ:

- Phân phối luồng dữ liệu thời gian thực theo cấu trúc phòng (Rooms) độc lập. Môi trường này hỗ trợ cơ chế xác thực riêng bằng mật khẩu dựa trên thuật toán băm PBKDF2.
- Xây dựng quy trình truyền tải tệp tin qua kiến trúc trạm trung chuyển (Server-Relayed Transfer). Quá trình này kết hợp cơ chế làm sạch dữ liệu (Sanitize) và cơ chế thu gom rác (Garbage Collection) nhằm giải phóng tài nguyên tự động.
- Tích hợp mô hình Kiểm soát truy cập phân quyền (Role-Based Access Control -

RBAC) nhằm quản lý các lệnh quản trị, bao gồm: kiểm soát phát ngôn (/mute), ngắt kết nối (/kick) và cấm truy cập (/ban).

4.2 Hướng phát triển tương lai

Nhằm đáp ứng các yêu cầu mở rộng kiến trúc và nâng cấp tính năng nghiệp vụ trong tương lai, một số định hướng phát triển được đề xuất bao gồm:

1. Nâng cấp tiêu chuẩn Bảo mật chuyển tiếp hoàn hảo (Perfect Forward Secrecy - PFS): Hệ thống hiện hành sử dụng thuật toán RSA trong quá trình phân phối khóa phiên AES, do đó chưa đáp ứng tiêu chuẩn PFS. Việc thay thế bằng giao thức trao đổi khóa Elliptic Curve Diffie-Hellman (ECDHE) sẽ thiết lập các khóa độc lập cho từng phiên giao tiếp. Kiến trúc này ngăn chặn rủi ro giải mã dữ liệu quá khứ trong trường hợp khóa bí mật dài hạn của máy chủ bị lộ.
2. Triển khai phương thức Mã hóa đầu cuối (End-to-End Encryption - E2EE): Bên cạnh luồng mật mã hóa Client-Server, việc áp dụng các tiêu chuẩn như Giao thức Signal (Signal Protocol) sẽ hỗ trợ tính năng mã hóa E2EE cho từng phiên giao tiếp. Ở mô hình này, máy chủ chỉ đóng vai trò trạm chuyển tiếp (Relay) đối với các gói tin đã được mã hóa, qua đó ngăn chặn khả năng truy cập hoặc thay đổi nội dung thông điệp tại máy chủ.
3. Chuyển đổi sang Kiến trúc Phân tán (Distributed Architecture): Nhằm đáp ứng khối lượng truy cập lớn vượt quá khả năng xử lý của một máy chủ độc lập, kiến trúc nguyên khối (Monolithic) có thể được tái cấu trúc theo mô hình phân tán (Cluster/Microservices). Việc tích hợp các hệ thống như Redis Pub/Sub hoặc Apache Kafka với vai trò Bộ điều phối thông điệp (Message Broker) giữa các máy chủ (Node) sẽ hỗ trợ cơ chế cân bằng tải và cải thiện khả năng mở rộng ngang (Horizontal Scalability) của hệ thống.
4. Xây dựng Giao diện người dùng (GUI) và Hỗ trợ truyền thông Đa phương tiện: Phát triển phân hệ máy khách từ giao diện dòng lệnh (CLI) lên giao diện đồ họa (GUI) thông qua các công nghệ như Qt (C++) hoặc Electron/Tauri. Đồng thời, dự án có thể tích hợp tiêu chuẩn WebRTC nhằm triển khai tính năng truyền tải âm thanh và hình ảnh (Voice/Video Call) theo kiến trúc mạng ngang hàng (P2P), từ đó giảm thiểu tải băng thông tại máy chủ trung tâm.